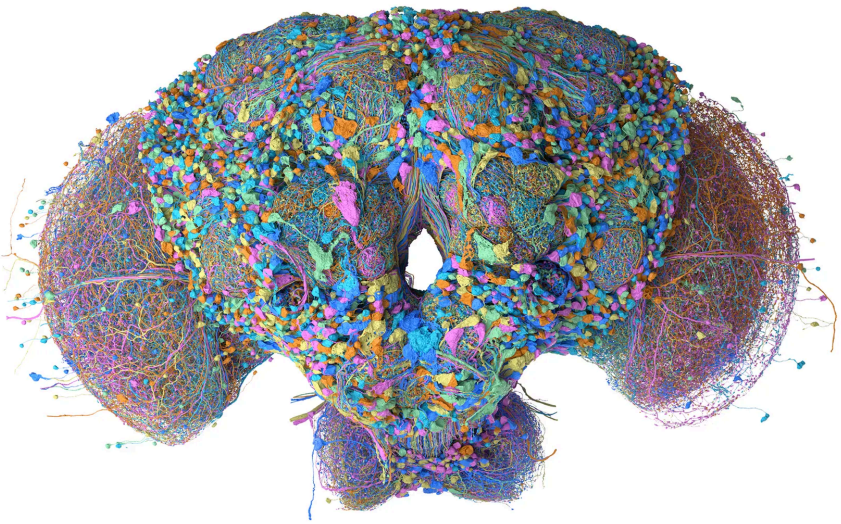
Introduction aux réseaux de neurones artificiels

CSI 4506 - automne 2026

Marcel Turcotte
Version : 18 oct. 2026 16h56

Préambule

Message du jour



5 amazing visuals show how the male fruit fly's brain map is advancing neuroscience, Google Research, 2026-09-03.

Des chercheurs du Janelia Research Campus du HHMI, de Google Research et d'établissements partenaires ont produit la première carte complète du cerveau d'une drosophile mâle. Elle comprend plus de 166 000 neurones.

Comme le décrit l'article ci-dessous, des développeurs ont créé en moins d'une journée des démonstrations de jeux vidéo en transmettant les entrées des jeux aux circuits neuronaux existants de la mouche, issus de l'évolution biologique. Ces démonstrations ont permis à la mouche de jouer à Minecraft, Doom, Beat Saber et à d'autres jeux, tout en permettant aux développeurs d'examiner comment le réseau traitait ces entrées et y réagissait.

- Scientists mapped a fruit fly's brain. The internet made it play Doom, par Antonia Davison, IBM Think news, 2026-09-22.

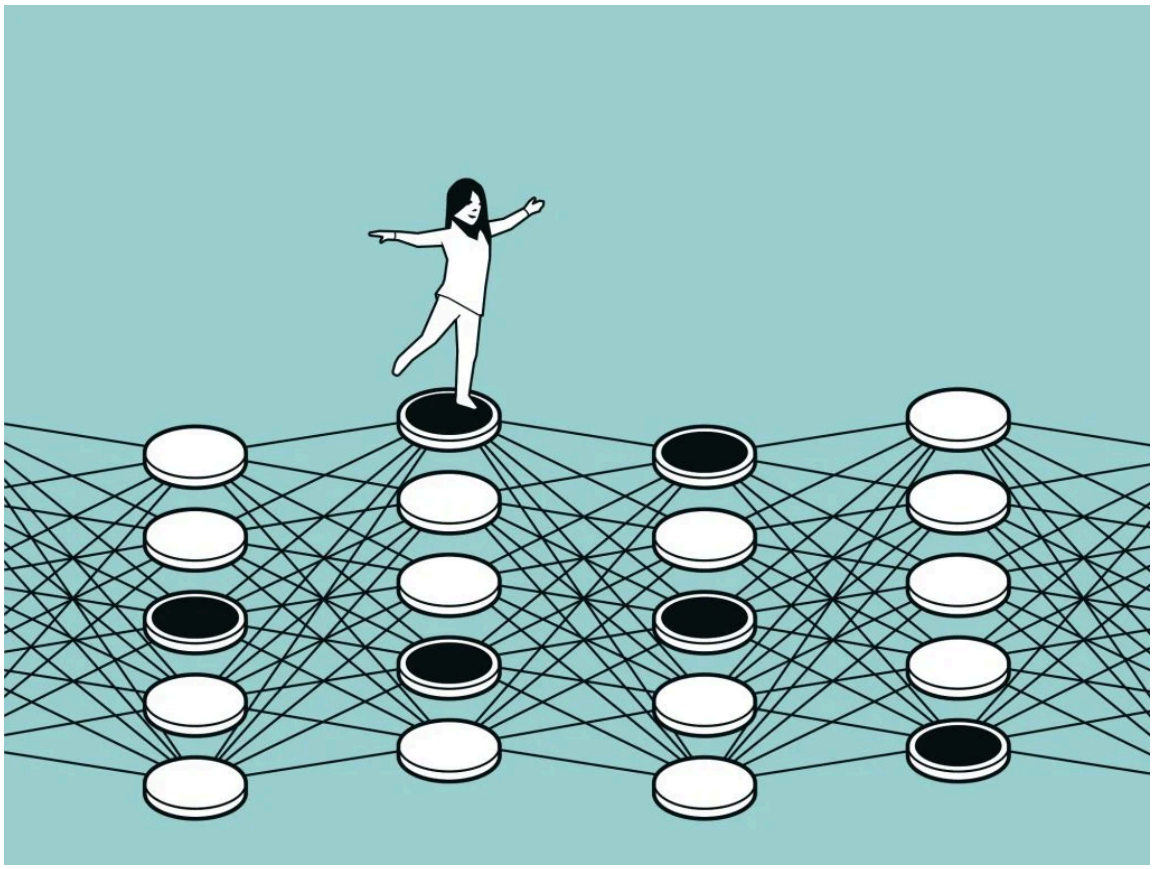
Message du jour

`x.com/evnsnclr/status/2095975490708291948`

Message du jour (suite)

`x.com/lyraaaa/status/2097527368919470162`

Message du jour (2024)



Il y a deux ans, cette présentation arrivait à point nommé : John J. Hopfield et Geoffrey E. Hinton venaient de recevoir le prix Nobel de physique 2024 « pour leurs découvertes et inventions fondamentales permettant l'apprentissage automatique au moyen de réseaux de neurones artificiels ».

Objectifs d'apprentissage

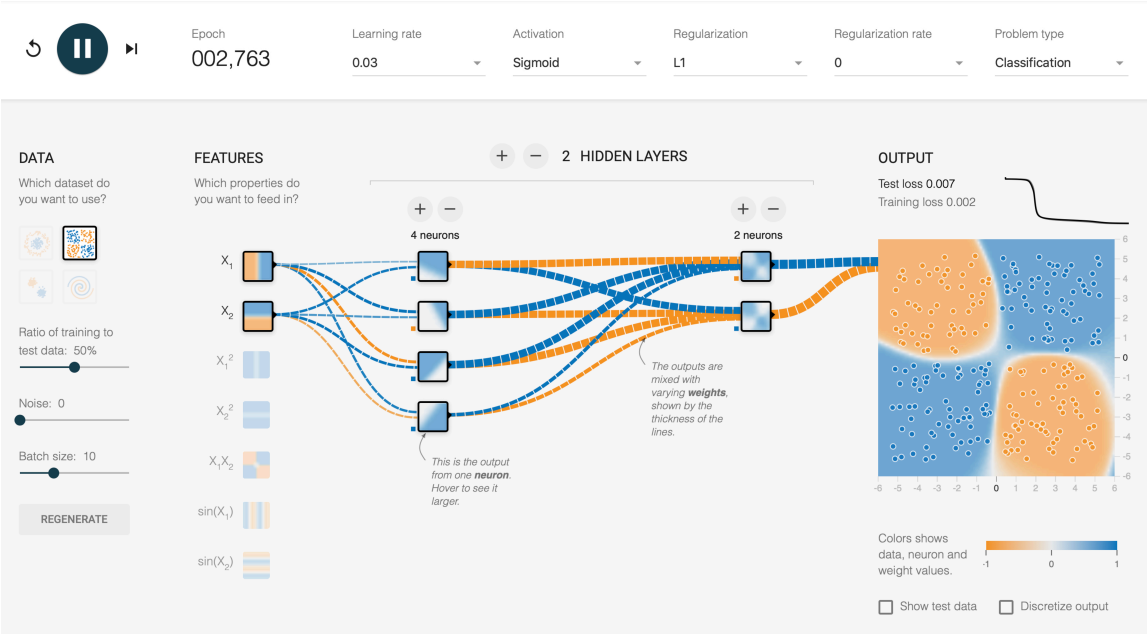
- **Expliquer** comment les poids, les biais et les activations déterminent le calcul de propagation avant d'un réseau.
- **Expliquer** pourquoi des couches cachées avec des activations non linéaires peuvent représenter XOR.
- **Comparer** la forme et l'ensemble des valeurs de sortie de la sigmoïde, de tanh et de ReLU.
- **Énoncer** les hypothèses et les limites du théorème d'approximation universelle.
- **Construire et évaluer** un classificateur Keras sur Fashion-MNIST avec un prétraitement cohérent et des ensembles d'entraînement, de validation et de test distincts.

Le cours a présenté l'IA symbolique et le connexionnisme comme deux grandes approches de l'intelligence artificielle. Les réseaux de neurones constituent un modèle connexionniste : de nombreuses unités de calcul simples travaillent ensemble pour associer des sorties aux entrées.

Deux questions guident cette introduction. Le **calcul** décrit ce que fait un réseau lorsque ses paramètres sont fixés. L'**apprentissage** décrit comment les données permettent de choisir ces paramètres. La compréhension du calcul vient d'abord; les présentations suivantes développent les méthodes d'entraînement.

Introduction

TensorFlow Playground



playground.tensorflow.org

TensorFlow Playground est une démonstration interactive de réseaux de neurones à propagation avant pour la classification en deux dimensions. Son schéma montre comment les entrées traversent les unités cachées pour produire une sortie, tandis que l'arrière-plan montre les régions de décision obtenues.

Les poids déterminent le calcul effectué par le réseau. Pendant l'entraînement, leur modification change les régions de décision. La distinction entre calculer une prédiction et apprendre les paramètres s'applique aussi aux réseaux plus grands utilisés plus loin dans cette présentation.

Expérimentez avec TensorFlow Playground pour comprendre intuitivement l'effet des transformations d'attributs et de l'architecture sur la classification. Pour les jeux de données circulaire et OU exclusif (XOR), qui ne sont pas linéairement séparables, essayez d'abord de construire des classificateurs sans couche cachée en ajoutant des attributs d'entrée. Ensuite, en utilisant seulement les attributs d'origine, explorez comment l'augmentation du nombre de couches et d'unités cachées permet au réseau d'apprendre des frontières de décision non linéaires.

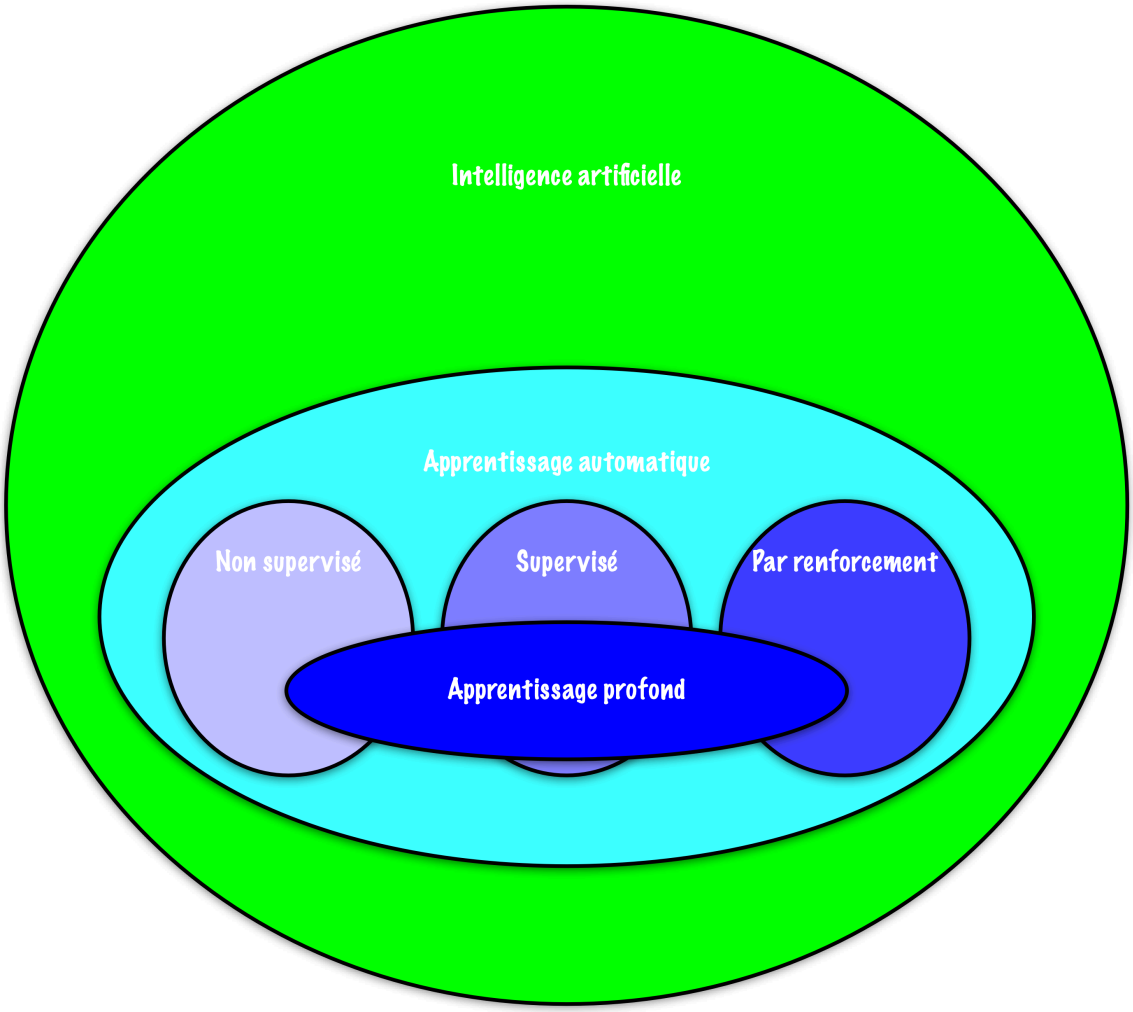
Réseaux de neurones (NN)

Les **réseaux de neurones artificiels** sont des modèles d'apprentissage automatique composés d'unités de calcul interconnectées, inspirées des neurones biologiques.

- **Calcul :** avec des paramètres fixés, un réseau associe des sorties aux entrées.
- **Apprentissage :** l'entraînement utilise des exemples pour ajuster les paramètres.

Aussi appelés **réseaux neuronaux** (*artificial neural networks* ou *neural nets*), abrégés en ANN ou NN dans la littérature anglophone.

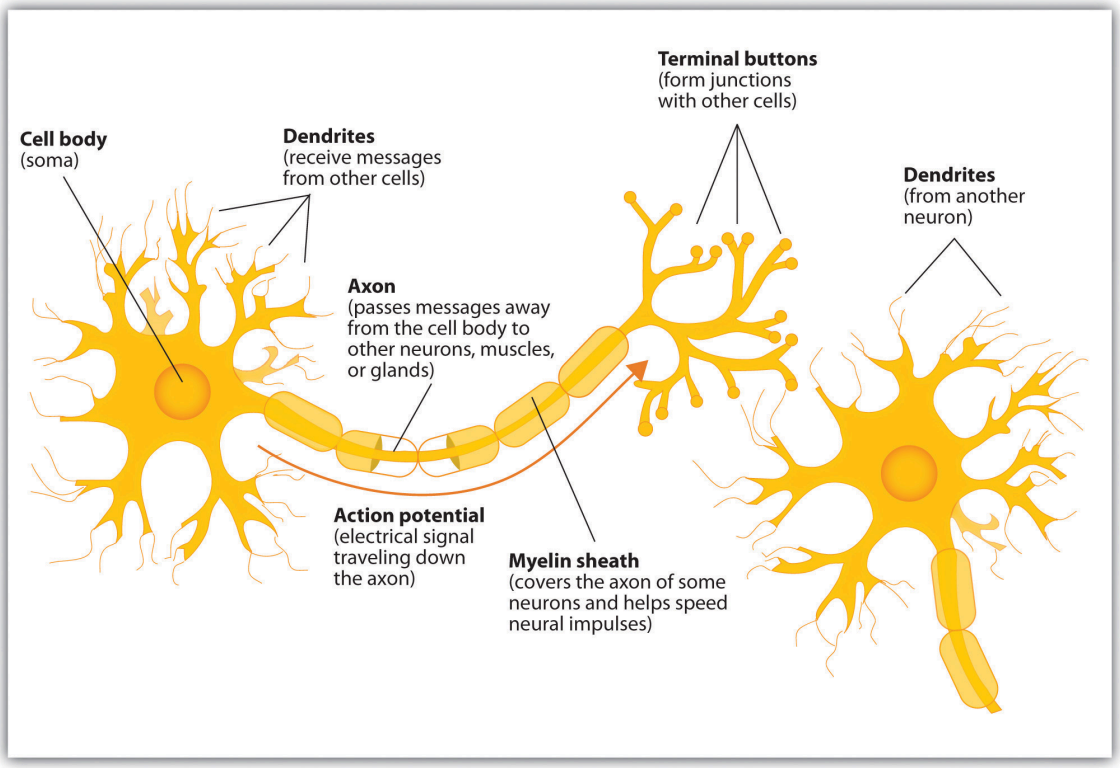
Problèmes d'apprentissage automatique



- **Apprentissage supervisé** : classification, régression
- **Apprentissage non supervisé** : autoencodeurs, apprentissage autosupervisé (*self-supervised learning*)
- **Apprentissage par renforcement** : les réseaux y occupent désormais une place importante

Nous commençons par l'apprentissage supervisé.

Un neurone



Attribution : Jennifer Walinga, CC BY-SA 4.0

Les neurones biologiques reçoivent des signaux par leurs connexions et transmettent des signaux à d'autres neurones. Les réseaux de neurones artificiels reprennent l'idée d'unités interconnectées, mais utilisent des calculs mathématiques délibérément simplifiés.

Pour les modèles étudiés ici, l'abstraction utile est une unité qui combine ses entrées et produit une sortie. La composition de ces unités forme un réseau qui effectue un calcul plus complexe.

Neurones interconnectés

https://youtu.be/uDnHOUPRTYM

Attribution : Molecular Mechanism of Synaptic Function, du Howard Hughes Medical Institute (HHMI). Vidéo publiée sur YouTube le 2018-11-15.

De la biologie, nous retenons surtout l'idée d'unités de calcul simples interconnectées pour former un réseau qui, dans son ensemble, effectue des calculs complexes.

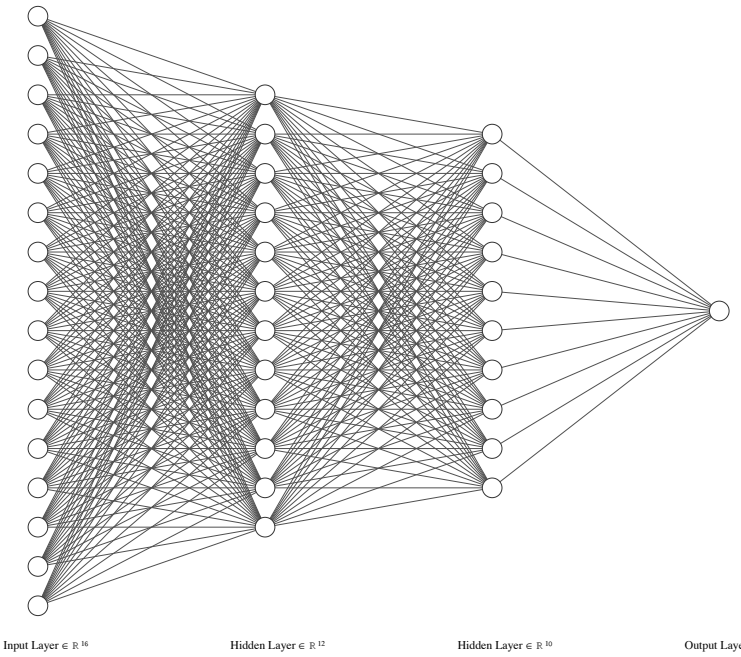
Bien que l'étude des réseaux de neurones biologiques soit importante, les réseaux de neurones artificiels ne reprennent qu'un nombre limité de concepts clés issus de ces recherches.

Neural Mind

> **Lakoff et Narayanan (2025)**
>
> **Les neurones ne sont que des cellules.** Seuls, ils sont incapables de penser. Mais, réunis en **circuits** reliés à notre corps de manière appropriée, ils ont permis aux êtres humains de survivre, de penser, de communiquer et de créer toute la culture.

Une unité combine ses entrées au moyen d'une somme pondérée et d'un biais, puis applique une fonction d'activation. Pour une unité à seuil binaire, l'équation $\mathbf{w}^T\mathbf{x} + b = 0$ définit un hyperplan de décision. Un réseau compose plusieurs de ces calculs : les unités intermédiaires transforment les entrées avant le calcul de la sortie.

Connexionnisme

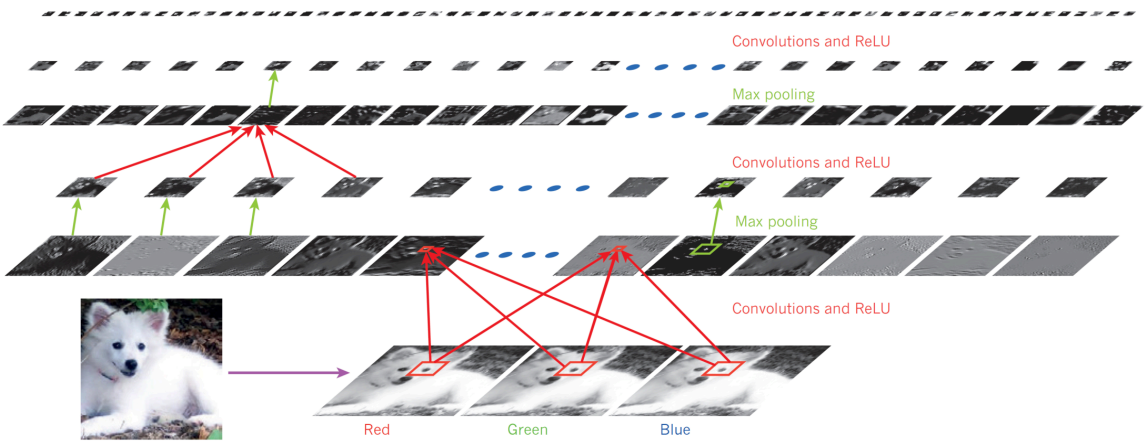


Attribution : LeNail, (2019). NN-SVG: Publication-Ready Neural Network Architecture Schematics. Journal of Open Source Software, 4(33), 747, https://doi.org/10.21105/joss.00747 (GitHub)

Le terme **connexionnisme** souligne les connexions entre les unités de calcul. Dans les réseaux à propagation avant étudiés ici, les unités sont organisées en couches et chaque connexion possède un poids.

L'architecture indique quelles unités sont connectées. Les poids et les biais déterminent la fonction calculée par cette architecture. L'entraînement ajuste ces paramètres à partir de données pour que les prédictions du réseau correspondent mieux aux cibles.

Hiérarchie des concepts



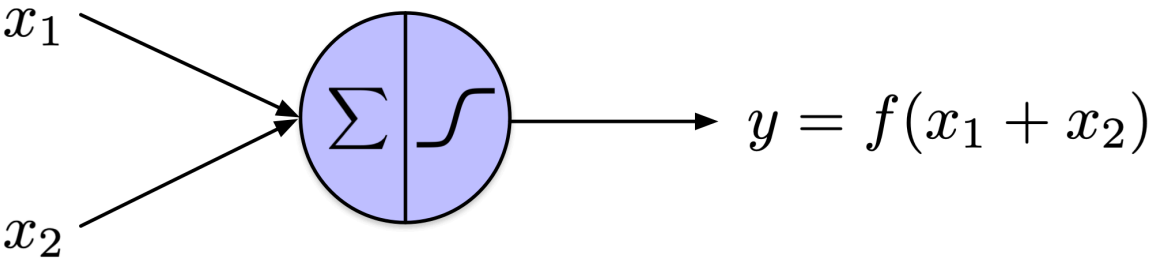
Attribution : LeCun et al. (2015)

Dans le livre *Deep Learning* (Goodfellow et al. 2016), Goodfellow, Bengio et Courville définissent l'apprentissage profond comme un sous-ensemble de l'apprentissage automatique qui permet aux ordinateurs de « comprendre le monde en termes d'une hiérarchie de concepts ».

Cette approche hiérarchique est l'une des contributions majeures de l'apprentissage profond. Elle réduit le besoin d'ingénierie manuelle des attributs et déplace l'attention vers la conception des architectures de réseaux de neurones.

Notions de base

Calcul avec des unités à seuil



où $x_1, x_2 \in \{0, 1\}$ et $f(z)$ est une **fonction de seuil** (*step function*) :

$$f(z) = \begin{cases} 0, & z < \theta \\ 1, & z \geq \theta \end{cases}$$

McCulloch et Pitts (1943) ont relié l'activité neuronale au **calcul booléen**.

McCulloch et Pitts ont étudié des réseaux de neurones idéalisés dont l'activité est binaire : une unité est active ou inactive. Leur modèle utilisait des seuils fixes ainsi que des connexions excitatrices et inhibitrices (McCulloch et Pitts 1943). Les exemples d'unités à seuil présentés ici illustrent de façon simplifiée le lien avec la logique.

La fonction $f(z)$ est l'indicatrice de la condition $z \geq \theta$: elle renvoie 1 lorsque la condition est satisfaite, et 0 sinon. Une unité associe donc une sortie binaire à des entrées binaires.

Des unités qui réalisent ET, OU et NON peuvent être composées en circuits calculant des fonctions booléennes. Dans ces exemples, les paramètres sont choisis à la main pour que le calcul soit facile à suivre.

Calcul avec des unités à seuil

$$f(x_1 + x_2) = \begin{cases} 0, & x_1 + x_2 < \theta \\ 1, & x_1 + x_2 \geq \theta \end{cases}$$

- Avec $\theta = 2$, l'unité réalise une porte logique **ET**.
- Avec $\theta = 1$, l'unité réalise une porte logique **OU**.

NON peut aussi être réalisé avec un poids d'entrée négatif et un seuil approprié.

Pour une entrée binaire x, une unité dont le poids d'entrée vaut -1 et le seuil vaut 0 calcule NON : $f(-x) = 1$ lorsque $x = 0$ et $f(-x) = 0$ lorsque $x = 1$. Un poids négatif ne suffit pas à définir une porte; son effet dépend du seuil et des autres entrées.

ET et OU ne nécessitent aucun apprentissage dans cette illustration : leurs seuils ont été choisis pour réaliser les tables de vérité souhaitées.

Les réseaux calculent des fonctions

- Des réseaux d'unités à seuil peuvent réaliser des **fonctions booléennes** en composant des opérations logiques.
- Lorsque les connexions et les seuils sont fixés, le réseau effectue un **calcul déterminé**.
- L'**apprentissage** pose une autre question : comment les données peuvent-elles déterminer les paramètres?

McCulloch et Pitts ont relié les réseaux de neurones idéalisés à l'étude mathématique du calcul (McCulloch et Pitts 1943). Dans les exemples présentés ici, la composition d'unités à seuil fournit un moyen concret de calculer une fonction booléenne.

Leur discussion plus générale du calcul de Turing comprenait des réseaux munis d'un ruban et de mécanismes de lecture et d'écriture. Un réseau à propagation avant fixé réalise une fonction choisie. L'idée clé de cette présentation est que des unités simples peuvent être composées en un réseau qui associe des sorties aux entrées.

Le perceptron de Rosenblatt déplace l'accent vers l'apprentissage à partir d'exemples : certains paramètres sont ajustés à partir de données plutôt que spécifiés entièrement à la main.

Perceptron

https://youtu.be/cNxadbrN_aI

L'engouement pour l'intelligence artificielle n'est pas nouveau : elle suscitait déjà beaucoup d'enthousiasme en 1958, comme l'illustre cette citation du *New York Times*.

> **8 juillet 1958, New York Times**
>
> La Marine a révélé aujourd'hui l'embryon d'un ordinateur électronique dont elle prévoit qu'il pourra **marcher**, **parler**, **voir**, **écrire**, **se reproduire** et être **conscient** de son existence.

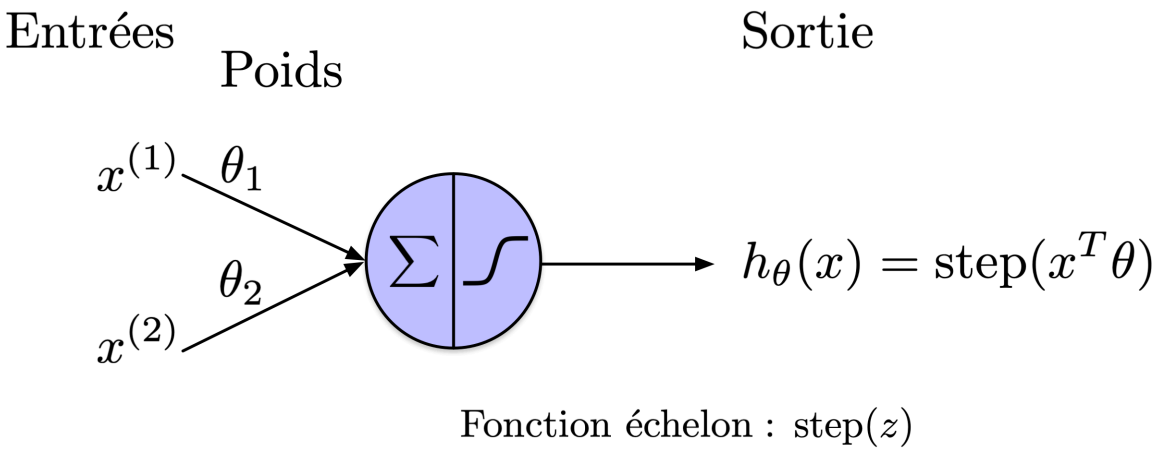
- The Machine That Changed the World - TV Series - 1992

Les images d'archives illustrent à la fois l'enthousiasme initial pour les machines capables d'apprendre et l'écart entre les démonstrations et les prédictions ambitieuses sur leurs capacités futures.

Perceptron

https://www.youtube.com/watch?v=Suevq-kZdIw

Unité logique à seuil



Rosenblatt (1958)

Le perceptron de Rosenblatt a introduit un modèle capable d'apprendre à partir d'exemples en ajustant ses paramètres (Rosenblatt 1958). Une unité logique à seuil fournit le modèle de calcul simplifié utilisé ici.

Ses entrées peuvent être réelles, mais sa sortie est discrète. Avec la fonction de Heaviside, la sortie vaut 0 ou 1. Chaque connexion d'entrée possède un poids.

La partie gauche du neurone, représentée par le symbole sigma, calcule une somme pondérée de ses entrées, exprimée par $\theta_1 x^{(1)} + \theta_2 x^{(2)} + \ldots + \theta_D x^{(D)} + b$.

Cette somme traverse ensuite une fonction de seuil, représentée dans la partie droite du neurone, pour produire la sortie.

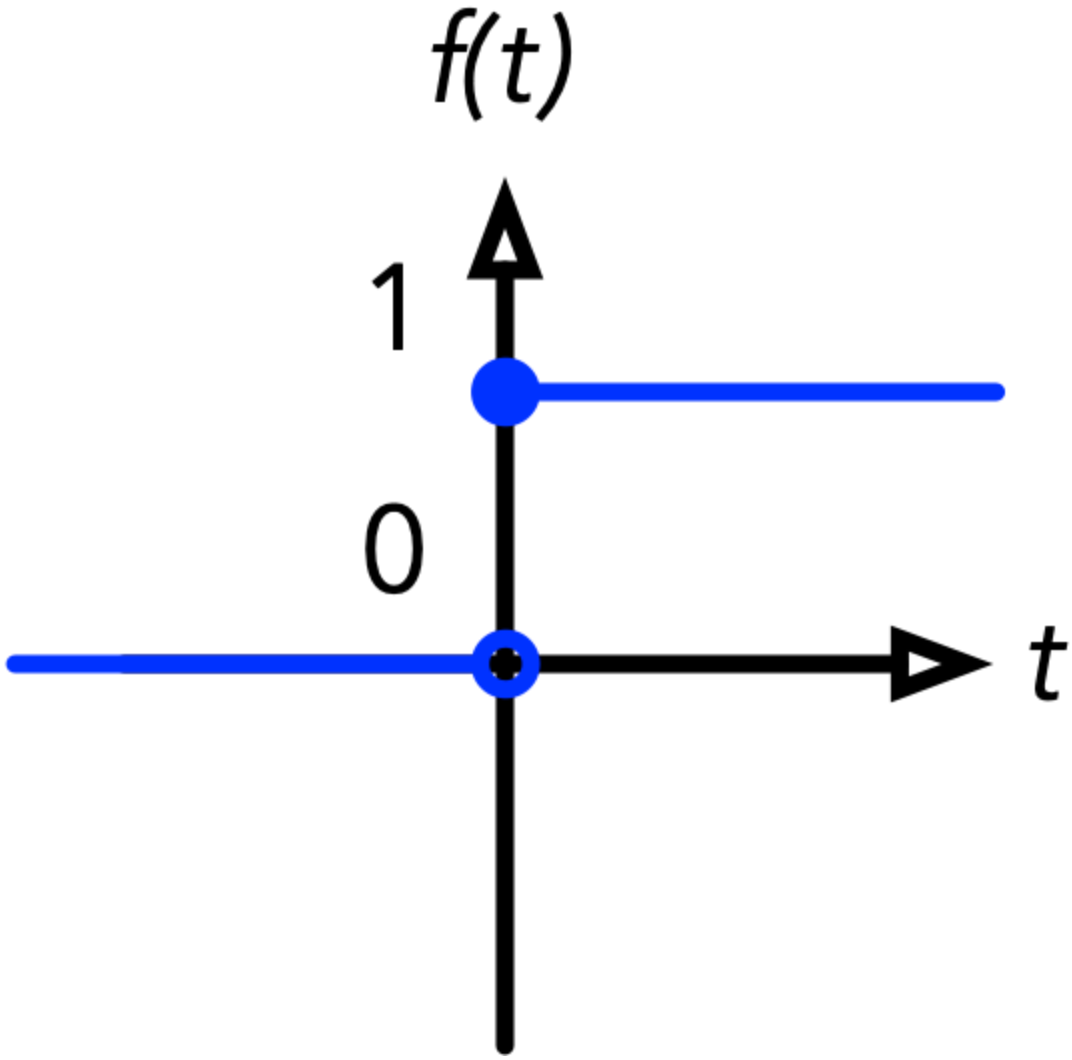
Ici, x et θ sont des vecteurs colonnes. La transposée x^T est un vecteur ligne; $x^T\theta$ est donc leur produit scalaire.

Le produit scalaire $x^T\theta$ est un scalaire donné par :

$$x^T\theta = x^{(1)}\theta_1 + x^{(2)}\theta_2 + \cdots + x^{(D)}\theta_D$$

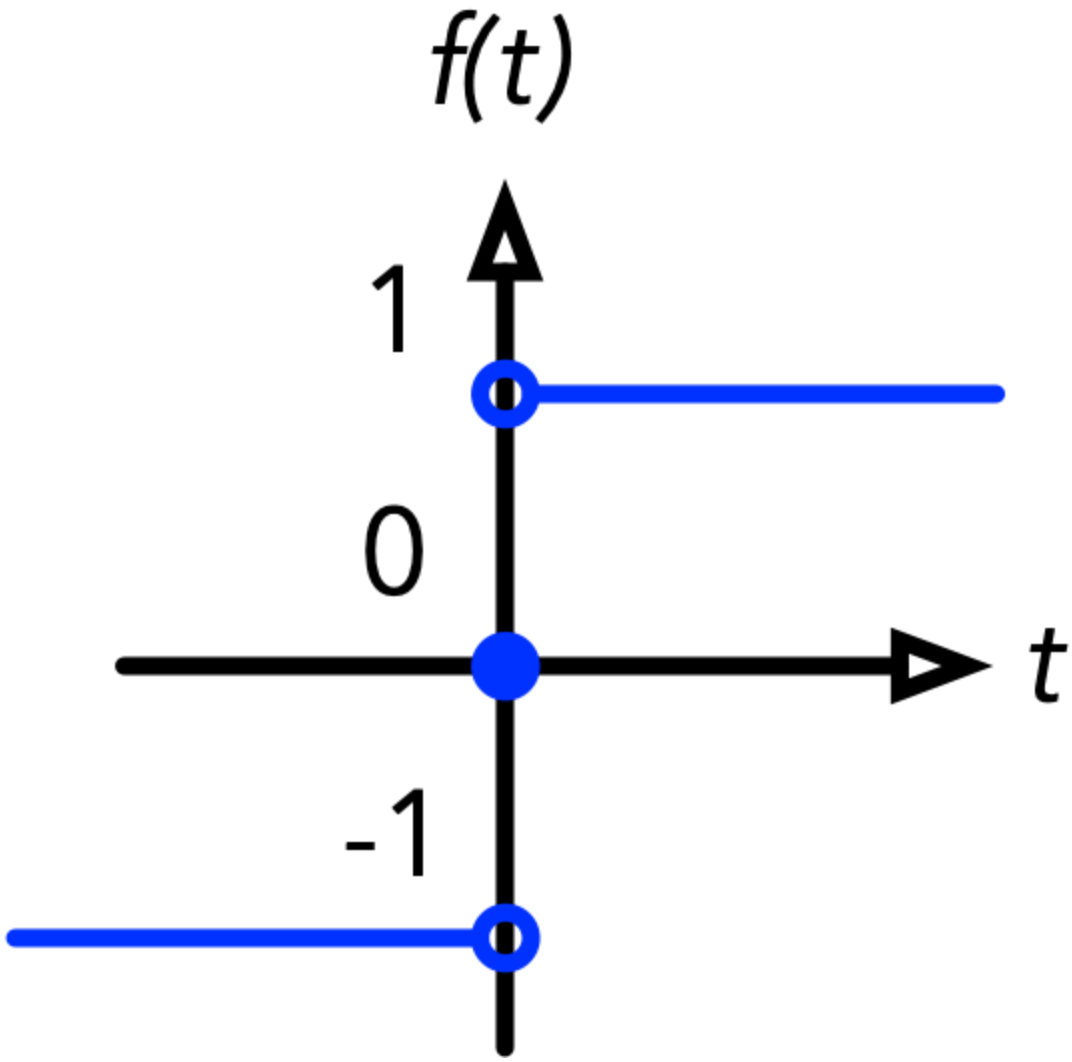
où $x^{(j)}$ et θ_j sont les composantes respectives des vecteurs x et θ. Les exposants de x désignent les attributs dans ces schémas.

Fonctions de seuil simples



$\text{Heaviside}(t)$ =

- 1, si $t \geq 0$
- 0, si $t < 0$

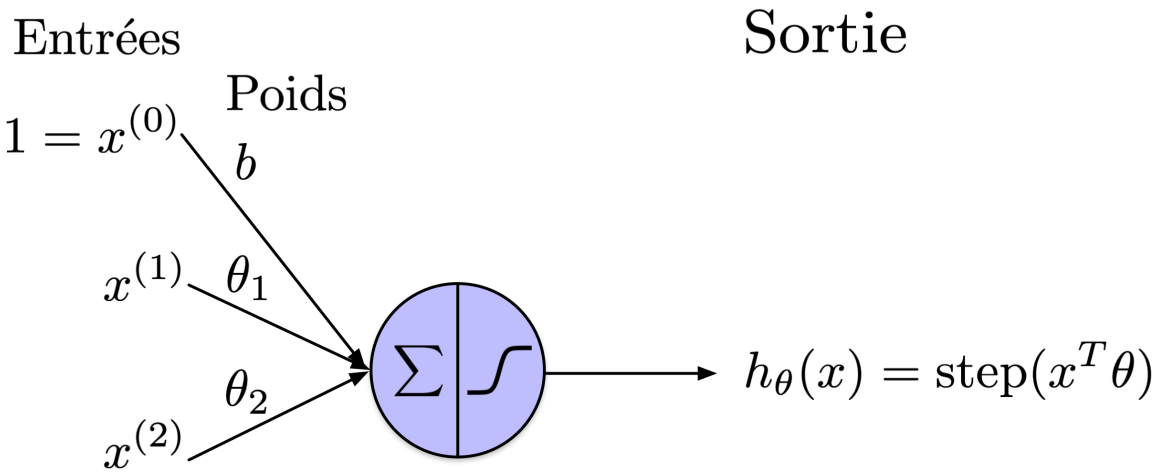


$\text{sign}(t)$ =

- 1, si $t > 0$
- 0, si $t = 0$
- -1, si $t < 0$

Les **fonctions de seuil** courantes comprennent la **fonction de Heaviside** (0 si l'entrée est négative, et 1 sinon) et la **fonction signe** (-1 si l'entrée est négative, 0 si elle est nulle, et 1 sinon).

Notation

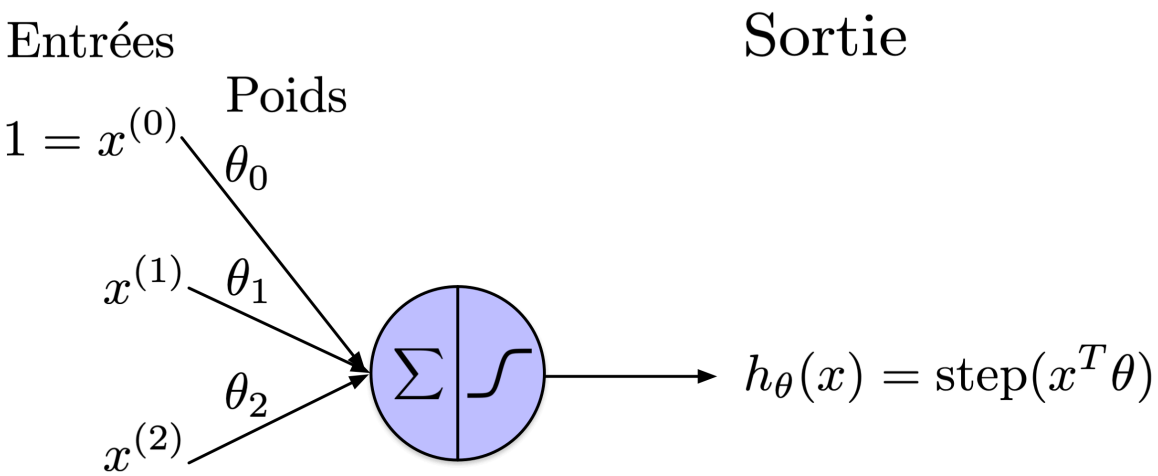


Fonction échelon : $\text{step}(z)$

Somme pondéré : $z = x^T\theta$

Ajoutez un attribut d'entrée de valeur constante 1. Associez-lui le poids $b = \theta_0$, où b est le biais, ou terme d'ordonnée à l'origine.

Notation



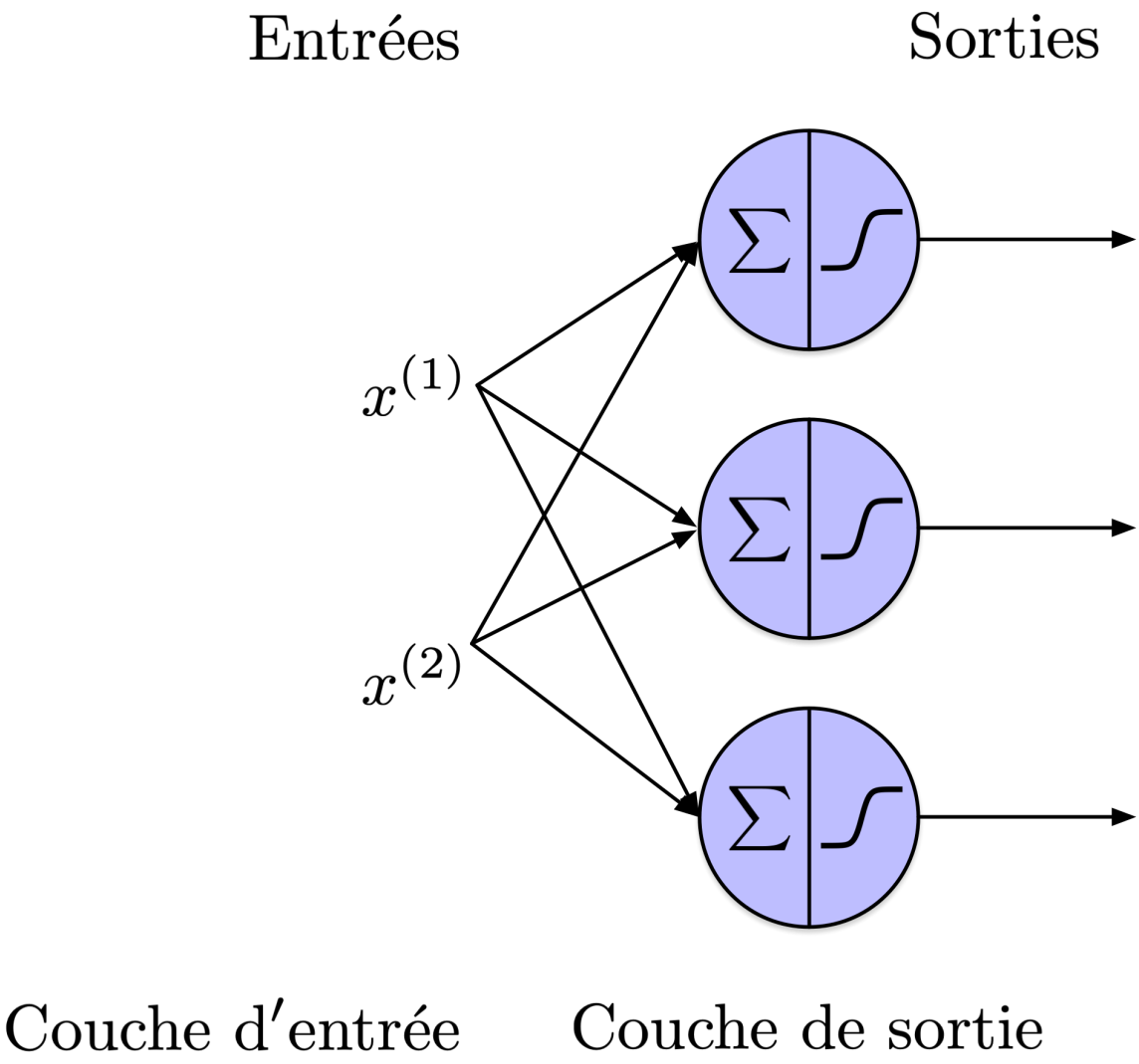
Fonction échelon : $\text{step}(z)$

Somme pondéré : $z = x^T\theta$

$\theta_0 = b$ est le biais.

La somme pondérée et le biais sont déjà connus en régression logistique. La régression logistique fait passer ce score dans une sigmoïde pour obtenir une probabilité; une unité à seuil binaire applique une fonction de seuil pour obtenir directement une décision de classe. Les deux modèles ont des frontières de décision linéaires dans leurs attributs d'entrée, mais leurs méthodes d'entraînement diffèrent.

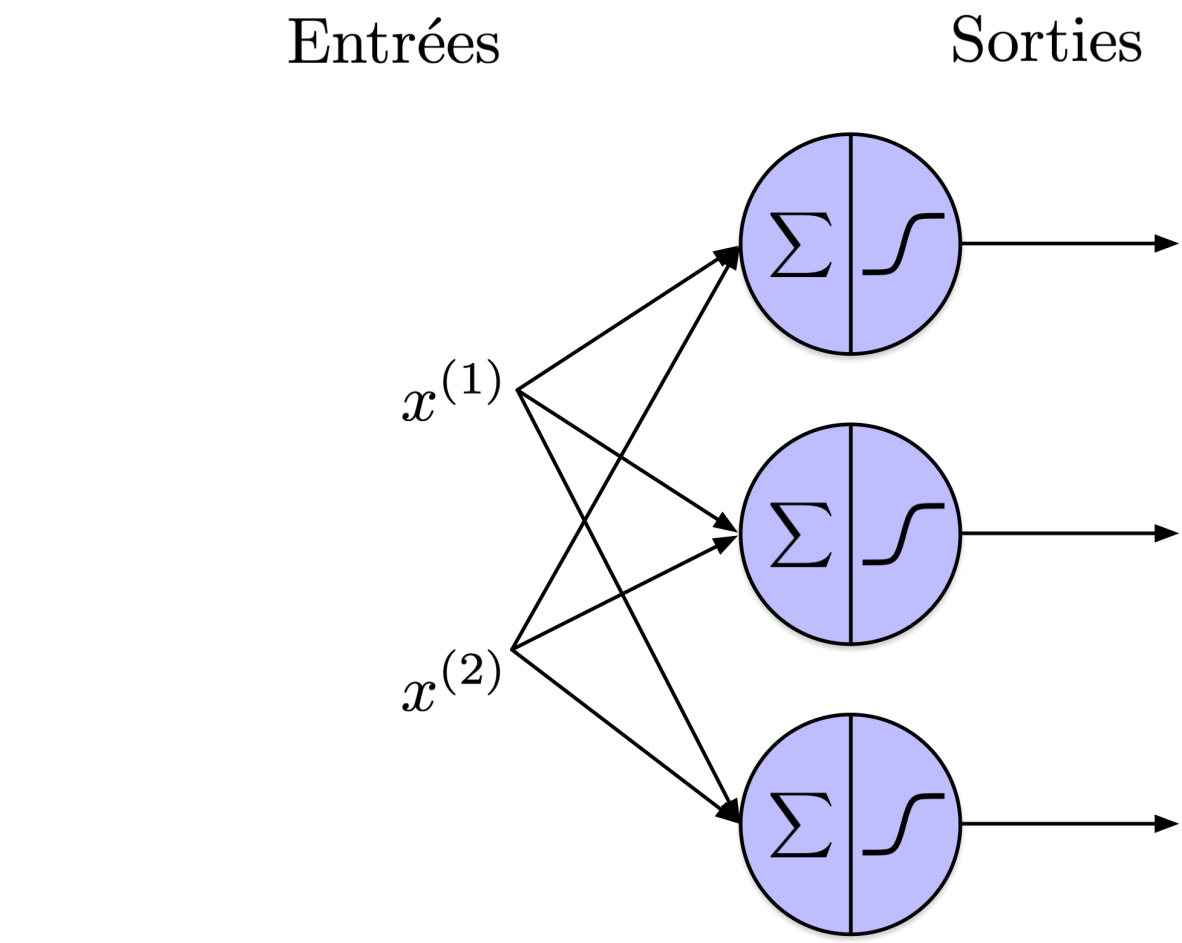
Perceptron



Un **perceptron** comprend une ou plusieurs **unités logiques à seuil** disposées en **une seule couche**, chaque unité étant connectée à toutes les entrées. Cette configuration est dite **entièrement connectée** ou **dense**.

Comme les unités de cette couche unique produisent aussi les sorties, on l'appelle **couche de sortie**.

Perceptron



Ce perceptron produit plusieurs sorties simultanément et effectue donc **plusieurs prédictions binaires** : il peut servir de **classificateur multiétiquette** (et peut aussi être utilisé pour la classification multiclasse).

Les tâches de classification peuvent être divisées en **classification multiétiquette** et **classification multiclasse**.

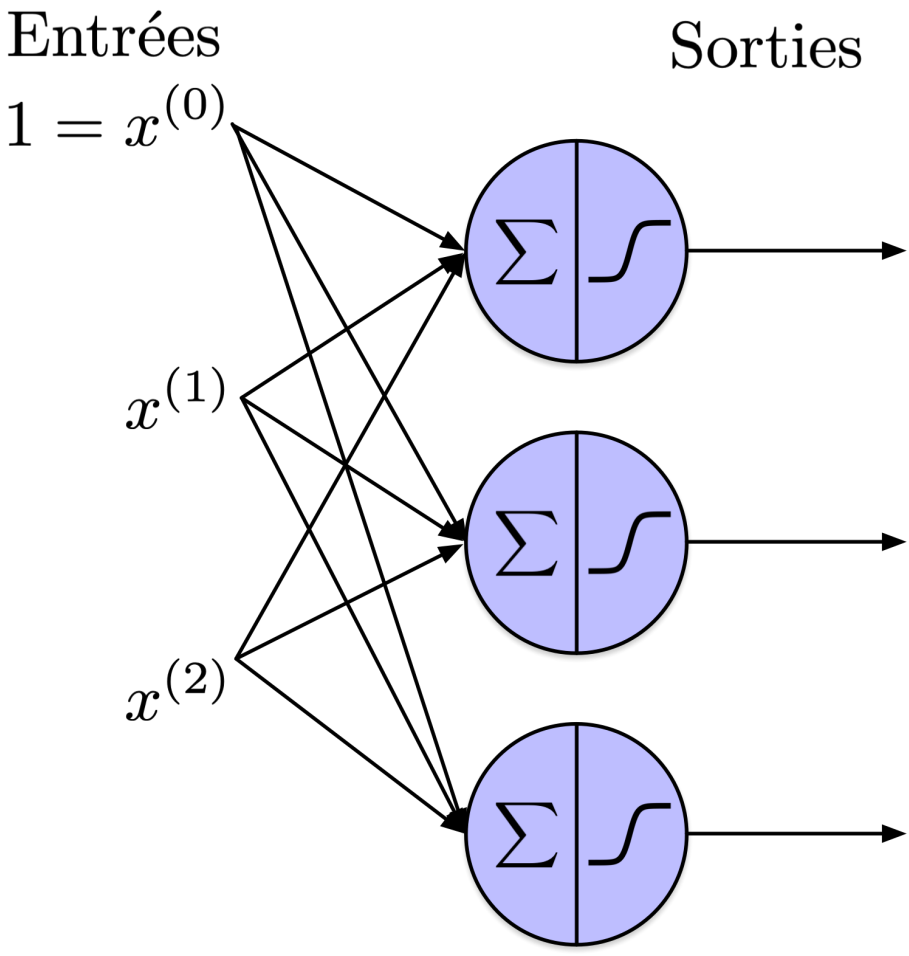
1. **Classification multiclasse :**
    - En classification multiclasse, chaque exemple est associé à une seule classe parmi au moins trois classes possibles. Les classes sont mutuellement exclusives : un exemple ne peut pas appartenir à plusieurs classes à la fois.
    - **Exemple :** classer une image comme représentant un chat, un chien ou un oiseau. Chaque image appartient à une seule de ces catégories.
2. **Classification multiétiquette :**
    - En classification multiétiquette, chaque exemple peut être associé à plusieurs classes simultanément. Les classes ne sont pas mutuellement exclusives : un

exemple peut appartenir à plusieurs classes à la fois.

- **Exemple :** annoter une image avec plusieurs attributs comme « extérieur », « coucher de soleil » et « plage ». L'image peut porter toutes ces étiquettes simultanément.

La différence clé concerne la relation entre les classes : la **classification multiclasse associe une seule étiquette à chaque exemple**, alors que la **classification multiétiquette peut lui en associer plusieurs**.

Notation



Comme précédemment, ajoutez un attribut d'entrée de valeur 1. **Attribuez un biais b à chaque neurone.** Chaque connexion entrante possède **implicitement** un poids.

Notation

- X est la **matrice des données d'entrée** : **chaque ligne correspond à un exemple** et **chaque colonne représente l'un des D attributs**.
- W est la **matrice des poids**, avec **une ligne par entrée (attribut)** et **une colonne par neurone**.
- Les **biais** peuvent être représentés séparément; les deux approches existent dans la littérature. Ici, b est un vecteur dont la **longueur est égale au nombre de neurones**.

Dans les réseaux de neurones, les **poids** sont souvent notés w (vecteur) ou W (matrice), plutôt que θ.

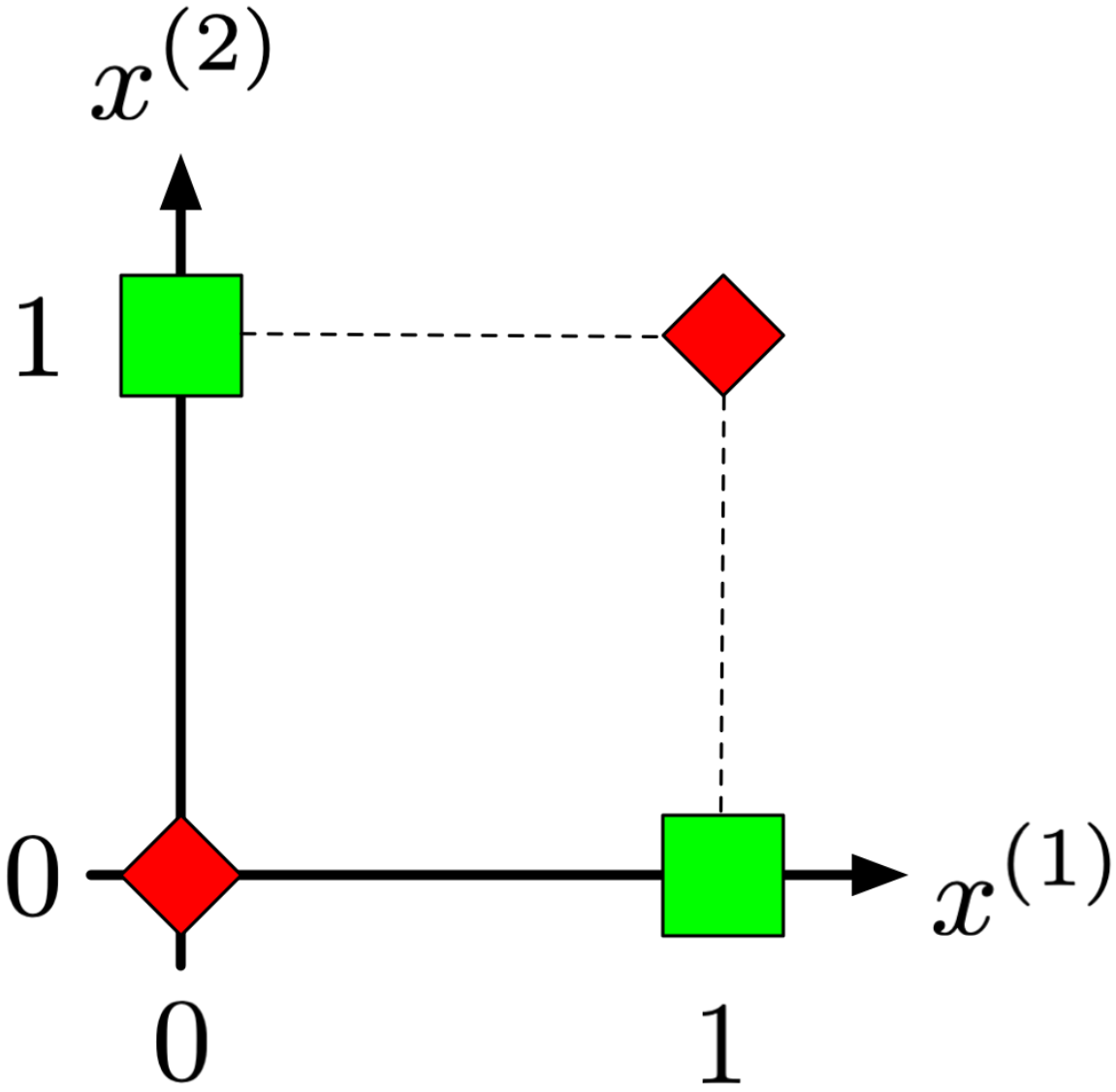
Du calcul à l'apprentissage

- La **propagation avant** utilise les poids et les biais actuels pour produire une prédiction.
- L'**entraînement** ajuste ces paramètres à partir d'exemples et de leurs sorties cibles.
- Le perceptron de Rosenblatt a marqué ce passage vers l'**apprentissage à partir des données**.

La distinction est déjà connue en régression logistique : évaluer un modèle dont les paramètres sont fixés et ajuster ces paramètres à partir de données sont deux opérations différentes. Cette présentation présente les calculs effectués par les réseaux de neurones; les deux présentations suivantes développent des méthodes d'entraînement des réseaux multicouches.

La règle d'apprentissage du perceptron sort du cadre de cette présentation. Pour les réseaux modernes, la rétropropagation calcule les gradients et un optimiseur, comme la descente de gradient stochastique, les utilise pour mettre à jour les paramètres.

Une limite : XOR



Une unité à seuil unique ne peut pas séparer les deux classes de XOR avec **une seule droite**. XOR n'est **pas linéairement séparable** dans l'espace des entrées d'origine.

Pour des entrées binaires, XOR vaut 1 en $(0, 1)$ et $(1, 0)$, et 0 en $(0, 0)$ et $(1, 1)$. Une unité à seuil calcule une somme pondérée plus un biais : sa frontière de décision dans cet espace bidimensionnel est donc une droite. Aucune droite ne sépare ces deux classes.

La régression logistique présente la même limite lorsqu'elle est appliquée directement à ces deux entrées. Des attributs non linéaires supplémentaires peuvent modifier ce qu'un classificateur linéaire peut représenter.

Dans *Perceptrons*, Minsky et Papert ont analysé les limites de calcul de modèles de perceptrons restreints (Minsky et Papert 1969). Ces résultats ne signifient pas que les réseaux multicouches sont incapables de représenter XOR. Représenter une fonction et apprendre des paramètres appropriés sont deux questions distinctes.

Composition de portes logiques

Au moins une entrée est vraie, et elles ne sont pas toutes les deux vraies :

$$A \oplus B = (A \vee B) \wedge \neg(A \wedge B)$$

- **OU** détecte si au moins une entrée est vraie.
- **ET** détecte si les deux entrées sont vraies.
- La combinaison de leurs sorties donne **XOR**.

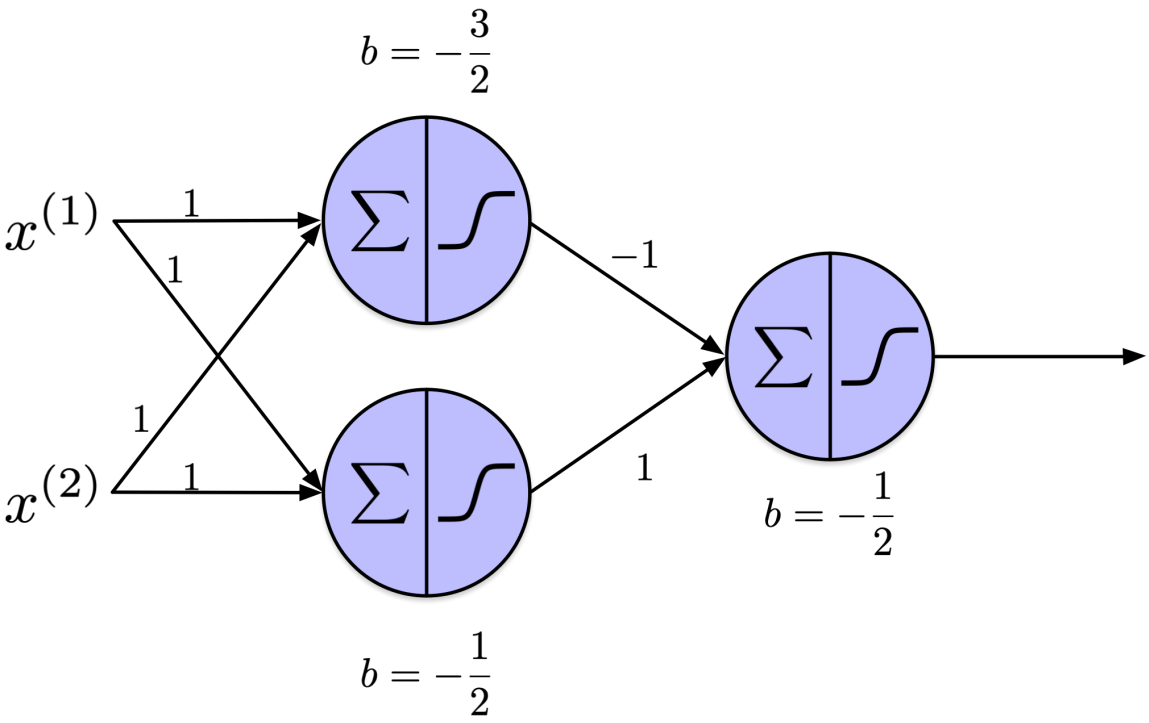
L'expression minimale ci-dessus correspond directement aux unités ET et OU du schéma du réseau. Deux autres expressions usuelles permettent la comparaison :

$$\begin{aligned} A \oplus B &= (A \wedge \neg B) \vee (\neg A \wedge B) \qquad (\text{disjunctive normal form}), \\ A \oplus B &= (A \vee B) \wedge (\neg A \vee \neg B) \qquad (\text{conjunctive normal form}). \end{aligned}$$

La première expression indique qu'exactement une entrée est vraie. Dans la seconde, $A \vee B$ exige au moins une entrée vraie, tandis que $\neg A \vee \neg B$ exclut le cas où les deux entrées sont vraies. Par la loi de De Morgan, $\neg A \vee \neg B = \neg(A \wedge B)$.

Un lien avec la logique booléenne : Geoffrey Hinton est l'**arrière-arrière-petit-fils de George Boole**, dont les travaux sont à la base de l'algèbre de Boole. Ce lien familial rapproche la logique de cet exemple de l'un des chercheurs qui ont contribué à populariser l'entraînement des réseaux multicouches. Projet du bicentenaire de George Boole de l'University College Cork

Perceptron multicouche



Un **perceptron multicouche** (MLP) comprend une ou plusieurs **couches cachées** entre ses entrées et sa couche de sortie. Ici, les unités cachées calculent **ET** (en haut) et **OU** (en bas).

Les nœuds d'entrée transmettent les attributs aux unités cachées. Celles-ci calculent des valeurs intermédiaires, qui deviennent les entrées de l'unité de sortie. Pour XOR, une couche cachée et une couche de sortie suffisent : il s'agit de deux couches de calcul, sans compter les nœuds d'entrée.

Cet exemple configuré à la main utilise des unités à seuil pour rendre les opérations logiques explicites. Les MLP modernes utilisent généralement des activations comme ReLU dans leurs couches cachées; leurs poids et leurs biais sont appris conjointement à partir de données.

XOR : calculs intermédiaires

A	B	**Cible** y	o_1 : **ET**	o_2 : **OU**	o_3 : **XOR**
0	0	0	0	0	0
0	1	1	0	1	1
1	0	1	0	1	1
1	1	0	1	1	0

L'unité de sortie calcule $o_2 \wedge \neg o_1$. La représentation cachée rend XOR **linéairement séparable**.

Dans le schéma, $A = x^{(1)}$ et $B = x^{(2)}$. Les sorties o_1 et o_2 désignent les unités cachées ET et OU, et o_3 est la prédiction du réseau. La table confirme que $o_3 = y$ pour les quatre combinaisons d'entrées.

Avec la convention $H(z) = 1$ pour $z \geq 0$ et $H(z) = 0$ sinon, ET et OU peuvent être réalisés par

$$o_1 = H(A + B - 1.5), \qquad o_2 = H(A + B - 0.5).$$

Une unité de sortie telle que $o_3 = H(o_2 - 2o_1 - 0.5)$ calcule $o_2 \wedge \neg o_1$. Le poids négatif supprime la sortie lorsque les deux entrées d'origine sont vraies. Plusieurs choix de poids et de biais produisent la même table de vérité.

Les entrées transformées (o_1, o_2) sont $(0, 0)$, $(0, 1)$ et $(1, 1)$. Les deux cas positifs de XOR sont associés au même point $(0, 1)$, qu'une droite peut séparer des cas négatifs. Le réseau calcule donc une représentation intermédiaire qui rend possible la classification finale.

Construction équivalente OU/NON-ET (NAND). Un poids négatif accompagné d'un biais approprié peut réaliser NON : $H(-A + 0.5) = \neg A$. De même, une seule unité à seuil réalise NON-ET par $H(-A - B + 1.5)$. Ses préactivations sont $1.5, 0.5, 0.5, -0.5$ pour les entrées $(0,0), (0,1), (1,0), (1,1)$, ce qui donne les sorties $1, 1, 1, 0$.

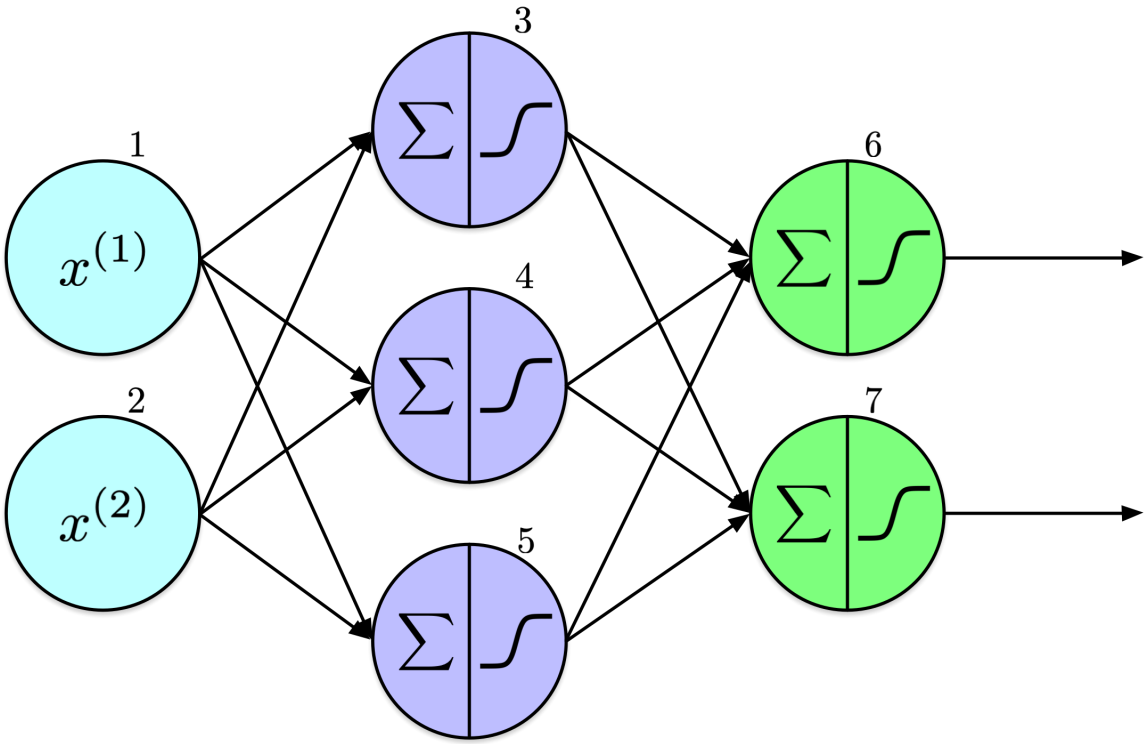
Les unités cachées peuvent donc aussi calculer $h_1 = H(A + B - 0.5)$ (OU) et $h_2 = H(-A - B + 1.5)$ (NON-ET). Une sortie ET, $H(h_1 + h_2 - 1.5)$, calcule alors XOR. Ce réseau est équivalent; le schéma affiché utilise plutôt des unités cachées ET et OU.

Représentation et apprentissage. Les poids de cet exemple sont fournis à la main. Dans une tâche générale, les sorties souhaitées des unités cachées ne sont pas fournies : l'entraînement doit déterminer comment la modification de leurs poids entrants affecte la prédiction finale. C'est le **problème d'attribution du crédit** (*credit assignment problem*).

Minsky et Papert ont établi des limites de représentation pour les modèles restreints qu'ils analysaient et ont exprimé leur scepticisme envers les extensions multicouches (Minsky et Papert 1969). Leurs résultats n'établissaient pas que tous les réseaux de neurones partagent ces limites. Ces résultats mathématiques sont distincts de la question historique plus large du déclin de l'intérêt et du financement de la recherche connexionniste. La préface de Bottou à la réédition de *Perceptrons* traite de cette distinction et du contexte historique.

Les fonctions de seuil discontinues constituent un obstacle supplémentaire à l'entraînement usuel par gradient : leurs dérivées sont nulles en dehors du seuil et non définies au seuil. Avec des fonctions d'activation appropriées, la rétropropagation permet de calculer la dépendance de la perte envers les paramètres des couches cachées. Rumelhart, Hinton et Williams ont contribué à populariser cette approche en 1986 (Rumelhart et al. 1986). La procédure d'apprentissage peut découvrir des attributs intermédiaires au lieu d'exiger qu'une personne les choisisse.

RN à propagation avant (FNN)



Dans cette architecture, l'information circule dans une seule direction, de gauche à droite, des entrées vers les sorties. On parle donc de **réseau de neurones à propagation avant** (*feedforward neural network*, FNN).

Le schéma montre les couches **d'entrée**, **cachée** et **de sortie**. Il contient deux nœuds d'entrée, trois unités cachées et deux unités de sortie. Si l'on ne compte que les couches qui effectuent des calculs paramétrés, il s'agit d'un **réseau à deux couches** : une couche cachée et une couche de sortie. On peut ajouter des couches et des unités cachées.

Il est souvent utile d'inclure explicitement des nœuds d'entrée qui n'effectuent aucun calcul, appelés **unités d'entrée** ou **neurones d'entrée**. Ces nœuds introduisent les attributs dans le réseau et les transmettent directement à la couche suivante, sans transformation. Dans le schéma, ce sont les nœuds bleu clair à gauche, numérotés 1 et 2. En général, **le nombre d'unités d'entrée correspond au nombre d'attributs**.

Les nœuds sont numérotés pour faciliter la discussion des poids qui les relient, comme $w_{1,5}$ entre les nœuds 1 et 5. De même, la sortie d'un nœud est notée o_k, où k est le numéro du nœud. Par exemple, pour $k = 3$, la sortie est o_3.

Somme pondérée et activation

Chaque unité de calcul effectue **deux opérations** :

$$z_j = \sum_{i \to j} w_{i,j} o_i + b_j, \qquad o_j = \varphi(z_j).$$

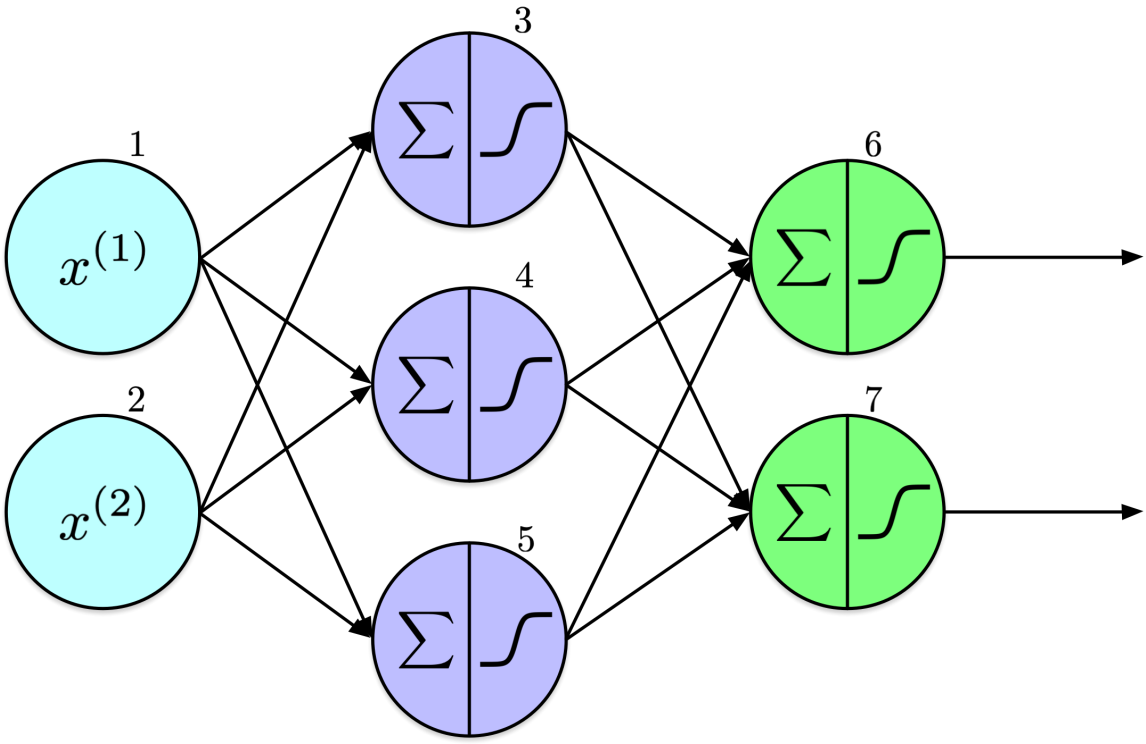
- z_j : **somme pondérée plus biais**, avant l'activation.
- o_j : **sortie** de l'unité, après application de l'activation φ.

La somme porte sur les nœuds connectés à l'unité j. Le poids $w_{i,j}$ relie le nœud source i au nœud destination j, et b_j est le biais de l'unité j. Les nœuds d'entrée transmettent les attributs sans modification : dans le schéma précédent, $o_1 = x^{(1)}$ et $o_2 = x^{(2)}$.

La quantité z_j est aussi appelée **préactivation** (*preactivation*). La sortie o_j est l'**activation**; certaines références la notent a_j. La notation o_j est conservée ici pour correspondre aux schémas et à l'exemple XOR.

Le symbole φ désigne une fonction d'activation générale. Lorsque l'activation est la sigmoïde logistique, elle est notée σ. Le symbole de sommation $\sum$, dans la moitié gauche d'une unité, représente l'agrégation : une opération différente de l'activation dans la moitié droite.

Propagation avant (calcul)



$$o_3 = \sigma(w_{1,3}x^{(1)} + w_{2,3}x^{(2)} + b_3)$$

$$o_4 = \sigma(w_{1,4}x^{(1)} + w_{2,4}x^{(2)} + b_4)$$

$$o_5 = \sigma(w_{1,5}x^{(1)} + w_{2,5}x^{(2)} + b_5)$$

$$o_6 = \sigma(w_{3,6}o_3 + w_{4,6}o_4 + w_{5,6}o_5 + b_6)$$

$$o_7 = \sigma(w_{3,7}o_3 + w_{4,7}o_4 + w_{5,7}o_5 + b_7)$$

Calculez **d'abord les sorties cachées**, puis utilisez-les pour calculer les **sorties finales**.

Chaque membre de droite forme d'abord une somme pondérée z_j, puis applique la sigmoïde pour obtenir o_j. Les unités de sortie dépendent des unités cachées : les sorties cachées doivent donc être disponibles avant de calculer les sorties finales.

Le schéma omet les biais pour rester lisible, mais chaque unité de calcul possède son propre biais. Un biais décale la préactivation de l'unité indépendamment de ses entrées. Pour une unité à seuil, il déplace la frontière de décision par rapport à l'origine; pour une unité sigmoïde ou ReLU, il déplace la position de la transition de l'activation. Ce rôle n'implique ni que l'entraînement ramènera automatiquement un biais inutile à zéro, ni que l'ajout de biais accélère toujours l'entraînement.

Ces deux sorties sigmoïdes sont des valeurs distinctes. Pour obtenir une distribution catégorielle sur des classes mutuellement exclusives, il faut une fonction de sortie comme le softmax utilisé dans l'exemple Fashion-MNIST.

Exemple numérique

In [2]:
```
import numpy as np
from scipy.special import expit as sigmoid

rng = np.random.default_rng(42)
x1, x2 = 0.5, 0.9

w13, w14, w15, w23, w24, w25 = rng.uniform(-1, 1, size=6)
w36, w46, w56, w37, w47, w57 = rng.uniform(-1, 1, size=6)
b3, b4, b5, b6, b7 = np.zeros(5)
```

In [3]:
```
z3 = w13 * x1 + w23 * x2 + b3
o3 = sigmoid(z3)
z4 = w14 * x1 + w24 * x2 + b4
o4 = sigmoid(z4)
z5 = w15 * x1 + w25 * x2 + b5
o5 = sigmoid(z5)
z6 = w36 * o3 + w46 * o4 + w56 * o5 + b6
o6 = sigmoid(z6)
z7 = w37 * o3 + w47 * o4 + w57 * o5 + b7
o7 = sigmoid(z7)

(o6, o7)
```

```
(np.float64(0.48638976530233274), np.float64(0.6254814695767328))
```

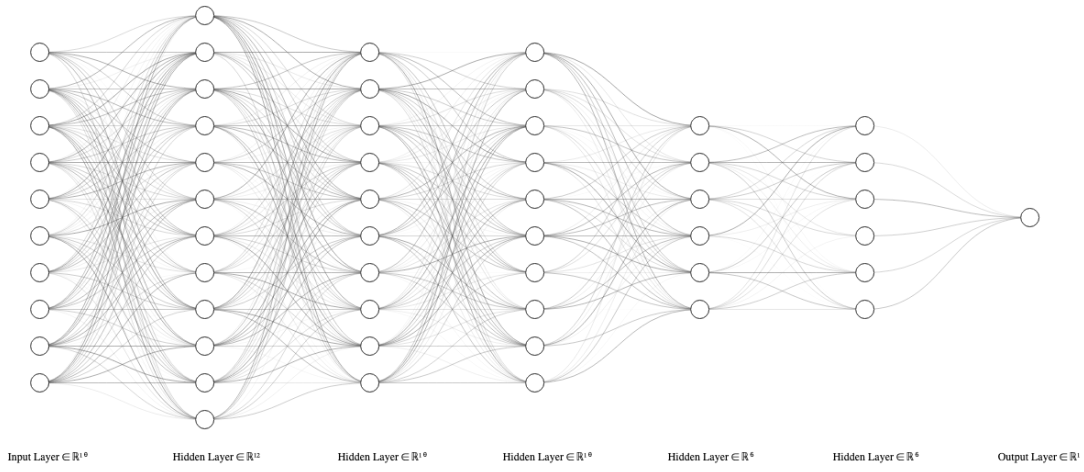
Exemple numérique de propagation avant.

Cet exemple applique les équations précédentes à une entrée. Dans le code, `w13` abrège $w_{1,3}$; `z3` et `o3` distinguent la préactivation et l'activation de l'unité 3. La graine aléatoire fixée rend les poids de cette illustration reproductibles.

L’exemple utilise des poids aléatoires et des biais nuls. Des poids initiaux différents permettent aux unités cachées de commencer par des calculs différents, ce qui brise la symétrie entre des unités autrement identiques. Les biais peuvent commencer à zéro et être appris malgré tout. Cette distinction est développée dans les présentations sur l’entraînement.

Les poids uniformes servent ici à une petite illustration et ne constituent pas une recommandation d’initialisation pour les réseaux profonds. Par défaut, les couches Dense de Keras utilisent des poids initialisés selon une loi uniforme de Glorot et des biais nuls. Documentation de Keras Dense

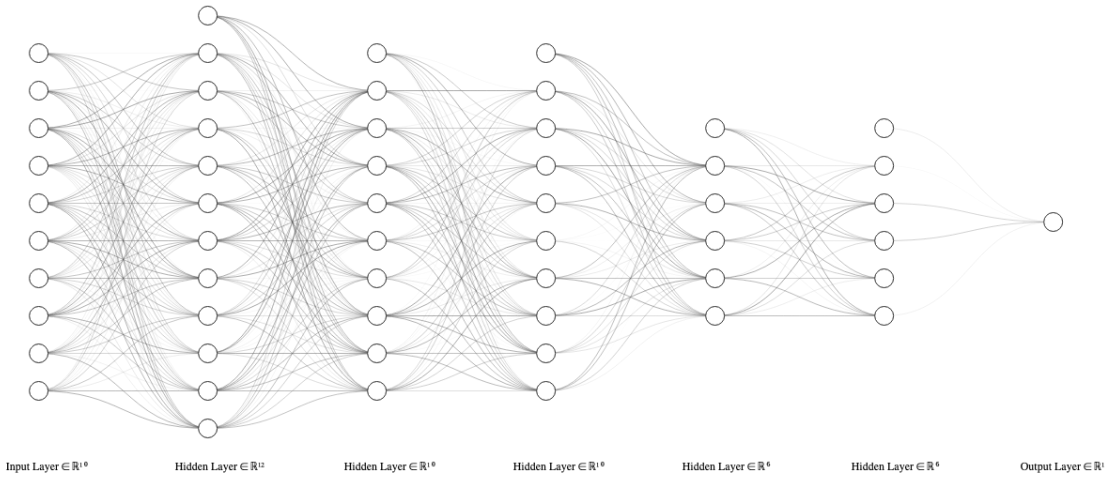
Propagation avant



La même succession de calculs s’applique à un **réseau de neurones profond**, qui possède davantage de couches cachées.

Produit avec **NN-SVG**, LeNail (2019).

Propagation avant avec biais



Le même réseau, avec les **entrées de biais affichées**.

Produit avec **NN-SVG**, LeNail (2019).

Pourquoi des activations non linéaires?

- Les couches cachées transforment leurs entrées en **représentations intermédiaires**.
- Sans activation non linéaire entre les couches, leur composition reste **une seule transformation affine**.
- Les activations non linéaires permettent au réseau de représenter des relations plus complexes entre les entrées et les sorties.

Une transformation affine est une somme pondérée plus un biais. En une dimension, la composition de $h = w_1 x + b_1$ et de $y = w_2 h + b_2$ donne

$$y = (w_2 w_1)x + (w_2 b_1 + b_2).$$

Il s'agit encore d'une transformation affine. Le même raisonnement s'applique aux calculs matriciels : empiler uniquement des couches affines ne permet donc pas au réseau de représenter XOR dans ses entrées d'origine. Les fonctions de seuil cachées de l'exemple XOR apportaient la non-linéarité nécessaire.

Les fonctions d'activation jouent deux rôles distincts. Leur non-linéarité détermine ce qu'un réseau peut représenter; leurs dérivées influencent la manière de l'entraîner. Cette présentation met l'accent sur le calcul et la représentation. La rétropropagation et la comparaison détaillée des dérivées sont développées dans la présentation 13.

Activations et entraînement

- La **rétropropagation** (*backpropagation*) calcule les gradients; un **optimiseur** les utilise pour mettre à jour les paramètres.
- Les dérivées d'une **fonction de seuil discontinue** ne fournissent pas de signal utile pour l'entraînement usuel par gradient.
- Les réseaux modernes utilisent des activations comme la **sigmoïde**, **tanh** et **ReLU**.

Une fonction de seuil discontinue a une dérivée nulle en dehors du seuil et n'est pas dérivable au seuil. La sigmoïde et tanh offrent des solutions lisses. ReLU est dérivable sauf en zéro, où les implémentations choisissent une convention; ce point isolé n'empêche pas la rétropropagation usuelle en pratique.

La non-linéarité et la possibilité d'entraîner le réseau sont deux questions distinctes : le seuil discontinu fournit déjà assez de non-linéarité pour le réseau XOR configuré à la main, même si ses dérivées usuelles ne conviennent pas à l'apprentissage des paramètres par descente de gradient.

Sigmoïde

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

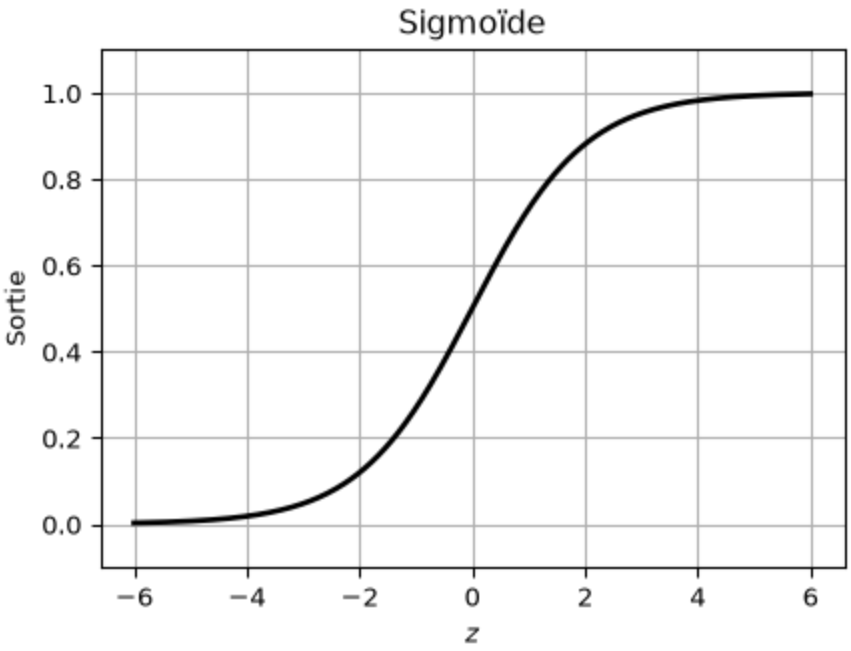
Ensemble des valeurs de sortie : $(0, 1)$.

In [4]:
```
import matplotlib.pyplot as plt


def relu(z):
    return np.maximum(0, z)


def plot_activation(activation, name, output_limits):
    z = np.linspace(-6, 6, 400)
    fig, ax = plt.subplots(figsize=(5, 3.5))
    ax.plot(z, activation(z), color="black", linewidth=2)
    ax.set(xlabel="$z$", ylabel="Sortie", title=name, ylim=output_limits)
    ax.grid(True)
    fig.patch.set_alpha(0)
    plt.show()


plot_activation(sigmoid, "Sigmoïde", (-0.1, 1.1))
```



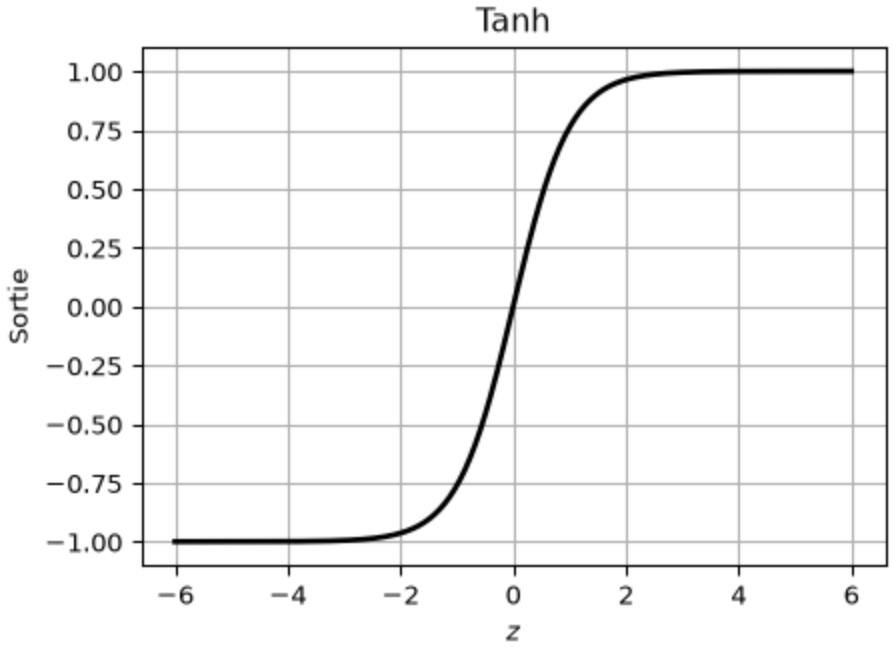
La sigmoïde est déjà connue en régression logistique. Elle associe à toute entrée réelle finie une valeur strictement comprise entre 0 et 1. Dans une couche cachée, cette valeur est une activation intermédiaire; elle ne représente pas automatiquement une probabilité de classe.

Tangente hyperbolique (tanh)

$$\tanh(z) = 2\sigma(2z) - 1$$

Ensemble des valeurs de sortie : $(-1, 1)$.

In [5]:
```
plot_activation(np.tanh, "Tanh", (-1.1, 1.1))
```



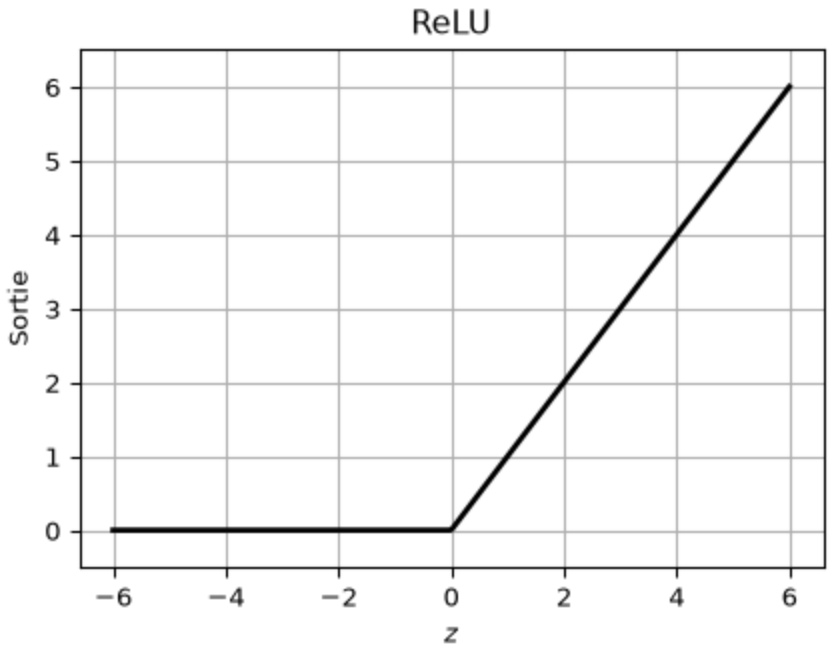
Contrairement à la sigmoïde, tanh peut produire des activations négatives et positives, avec $\tanh(0) = 0$. Son ensemble de valeurs de sortie est centré sur zéro, mais la moyenne réelle des activations d'une couche dépend toujours de ses entrées et de ses paramètres. Cette propriété ne garantit pas à elle seule une convergence plus rapide.

Unité linéaire rectifiée (ReLU)

$$\mathrm{ReLU}(z) = \max(0, z)$$

Ensemble des valeurs de sortie : $[0, \infty)$.

In [6]:
```
plot_activation(relu, "ReLU", (-0.5, 6.5))
```



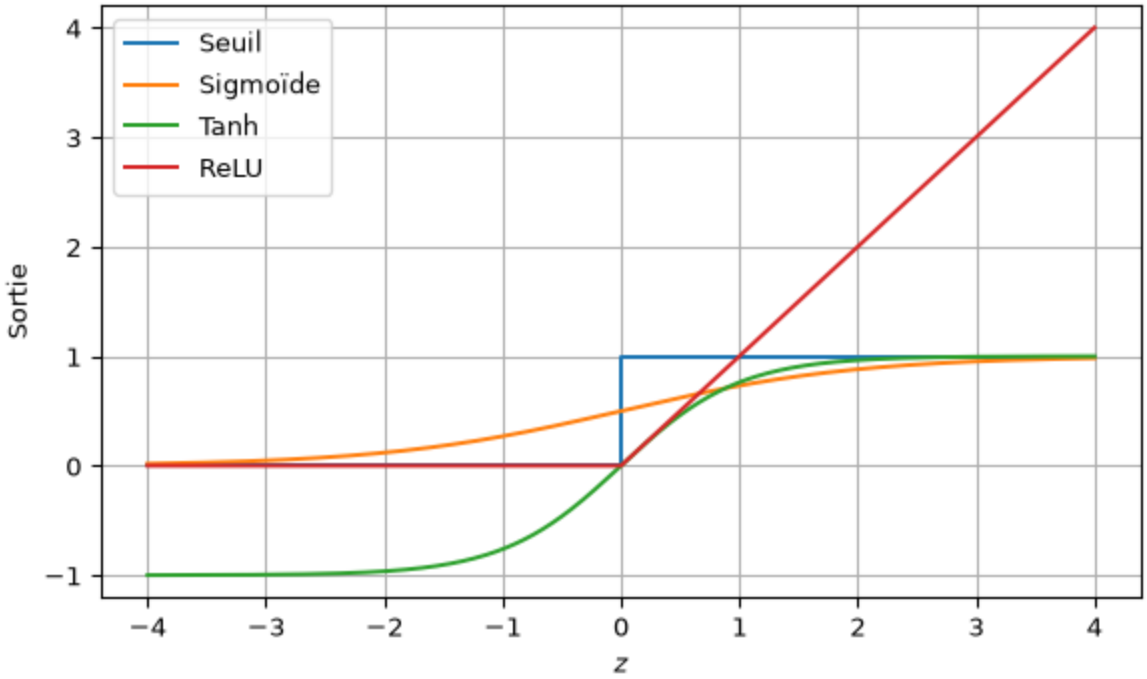
Un choix courant pour les **couches cachées**, y compris dans l'exemple Keras.

ReLU renvoie zéro pour les entrées négatives et transmet les entrées positives sans modification. Bien que chaque branche soit linéaire, le changement de pente en zéro rend la fonction non linéaire. La composition de transformations affines et de ReLU peut produire des fonctions affines par morceaux comportant de nombreuses régions.

ReLU est un choix courant pour les couches cachées, et non un choix universel pour toute couche ou toute tâche. Dans Keras, une couche Dense sans activation explicite utilise l'identité (activation linéaire). Les activations de sortie sont choisies en fonction de la tâche de prédiction.

Comparaison des fonctions d'activation

In [7]:
```
z = np.linspace(-4, 4, 401)
fig, ax = plt.subplots(figsize=(7, 4))
ax.step(z, (z >= 0).astype(int), where="post", label="Seuil")
ax.plot(z, sigmoid(z), label="Sigmoïde")
ax.plot(z, np.tanh(z), label="Tanh")
ax.plot(z, relu(z), label="ReLU")
ax.set(xlabel="$z$", ylabel="Sortie", ylim=(-1.2, 4.2))
ax.grid(True)
ax.legend()
plt.show()
```



La fonction de seuil produit une décision discrète. La sigmoïde et tanh produisent des activations continues bornées, tandis que ReLU est continue et non bornée supérieurement. Ces différences influencent les valeurs intermédiaires disponibles pour les couches suivantes.

Cette comparaison porte sur les fonctions elles-mêmes. Leurs dérivées et les conséquences sur la propagation des gradients sont examinées dans la présentation 13.

Approximation universelle

Définition

Le **théorème d'approximation universelle** (*universal approximation theorem*, UAT) établit que, sur un **domaine d'entrée compact**, un réseau ayant une seule couche cachée suffisamment large, des **activations sigmoïdes** et une **sortie linéaire** peut approximer toute **fonction cible continue** donnée avec la précision souhaitée.

Cybenko (1989); Hornik et al. (1989)

Un ensemble compact de $\mathbb{R}^n$ est à la fois **fermé** et **borné**. Un intervalle fermé comme $[-4, 2]$ en est un exemple simple.

Pour toute fonction cible continue sur ce domaine et toute tolérance d'erreur strictement positive, le théorème garantit l'existence d'un réseau fini dont l'erreur reste inférieure à cette tolérance en tout point du domaine. La largeur nécessaire et les valeurs des

paramètres peuvent dépendre de la cible et de la tolérance (Cybenko 1989; Hornik et al. 1989).

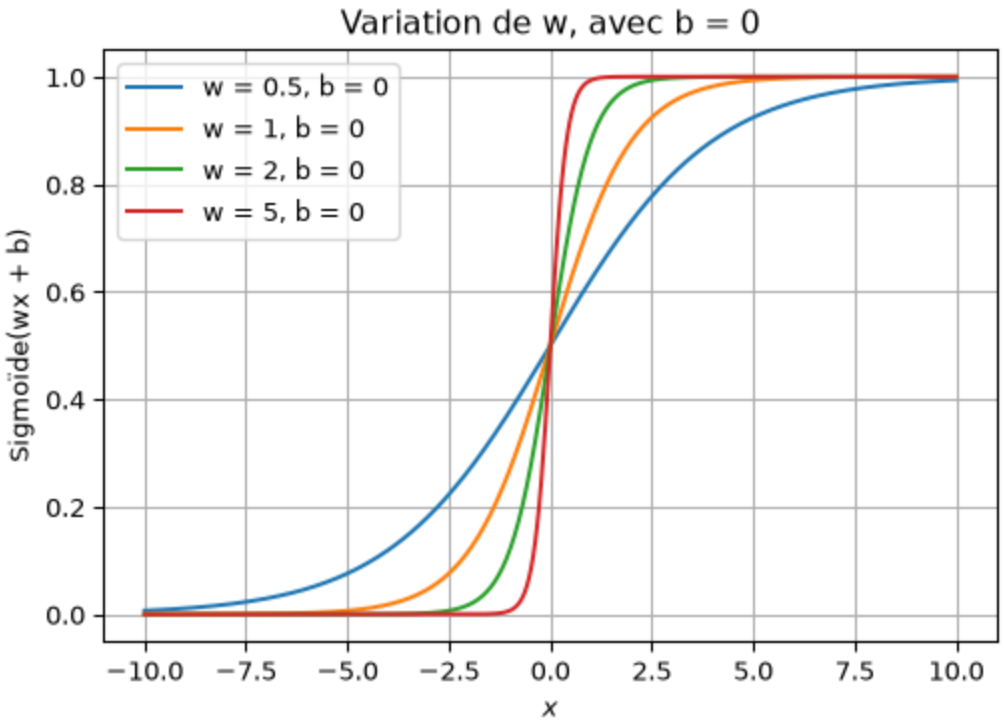
Il s'agit d'un **résultat d'existence concernant la représentation**. Il ne garantit ni qu'un algorithme d'entraînement trouvera les paramètres appropriés, ni que le réseau sera petit, ni que l'apprentissage à partir d'un échantillon fini généralisera bien. Il ne garantit pas non plus l'extrapolation hors du domaine spécifié.

La version avec sigmoïde fournit un énoncé classique commode. Des résultats plus généraux couvrent d'autres activations, dont ReLU, utilisée dans la démonstration par le code. La non-linéarité seule n'est pas une hypothèse suffisante sur l'activation. Leshno et al., *Multilayer feedforward networks with a nonpolynomial activation function can approximate any function*

Effet de w : sigmoïde

In [8]:
```
def plot_parameter_effect(activation, name, weights, biases, title):
    x = np.linspace(-10, 10, 400)
    fig, ax = plt.subplots(figsize=(6, 4))
    for w in weights:
        for b in biases:
            ax.plot(x, activation(w * x + b), label=f"w = {w}, b = {b}")
    ax.set(xlabel="$x$", ylabel=f"{name}(wx + b)", title=title)
    ax.grid(True)
    ax.legend()
    plt.show()

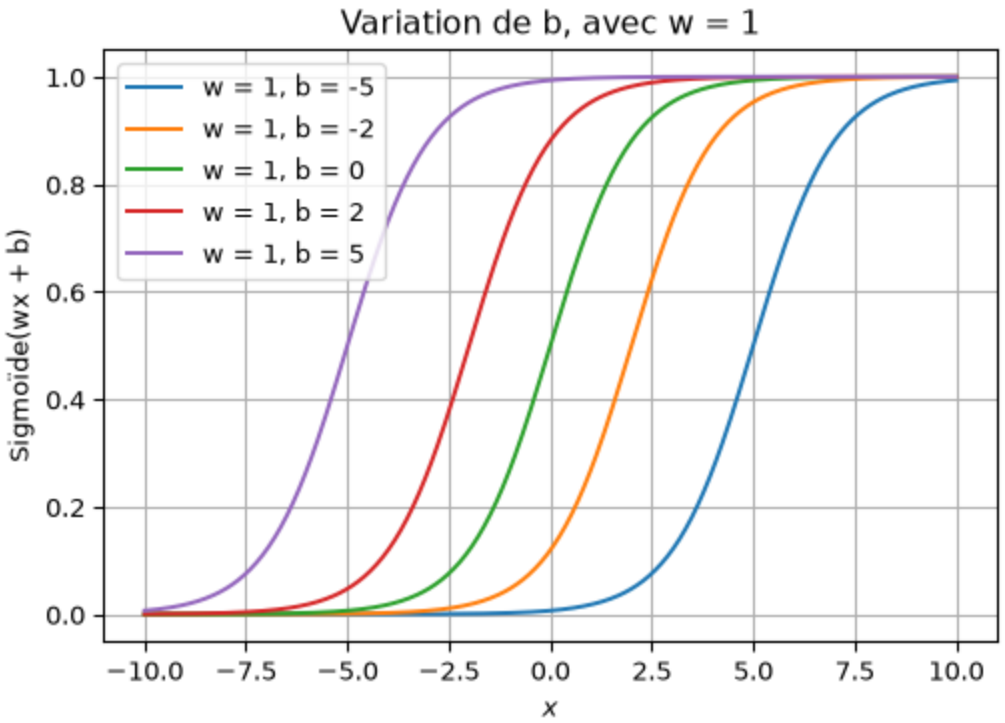

plot_parameter_effect(sigmoid, "Sigmoïde", weights=[0.5, 1, 2, 5], biases=[0
                      title="Variation de w, avec b = 0")
```



Le poids modifie la **raideur** de $\sigma(wx + b)$.

Effet de b : sigmoïde

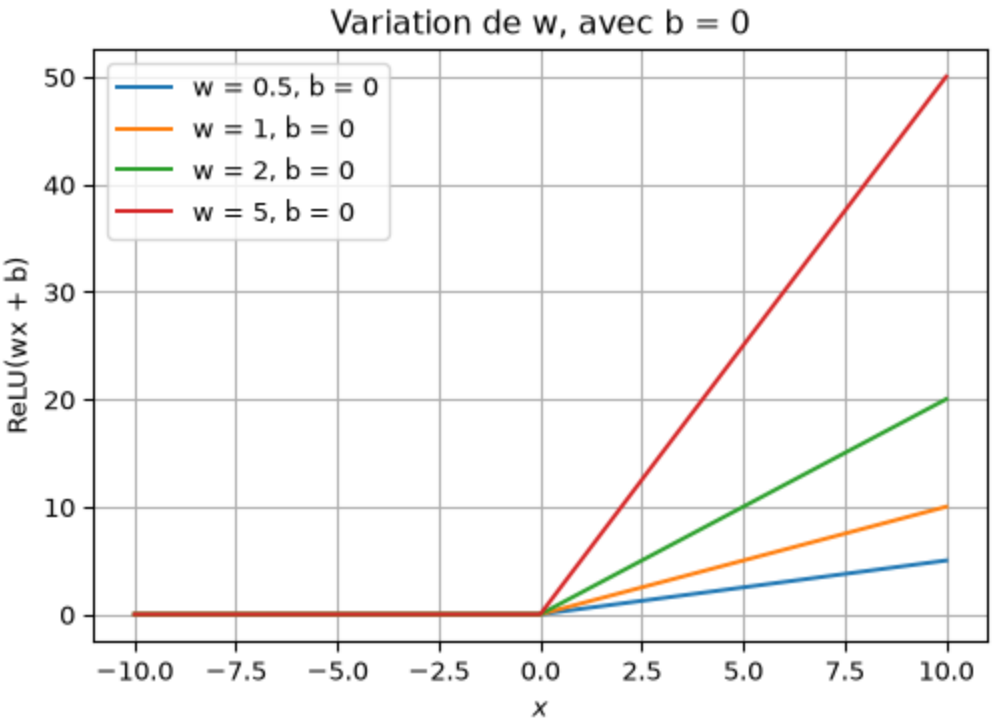
In [9]:
```
plot_parameter_effect(
    sigmoid, "Sigmoïde", weights=[1], biases=[-5, -2, 0, 2, 5],
    title="Variation de b, avec w = 1"
)
```



Pour $w \neq 0$ fixé, le biais déplace la transition en $x = -b/w$.

Effet de w : ReLU

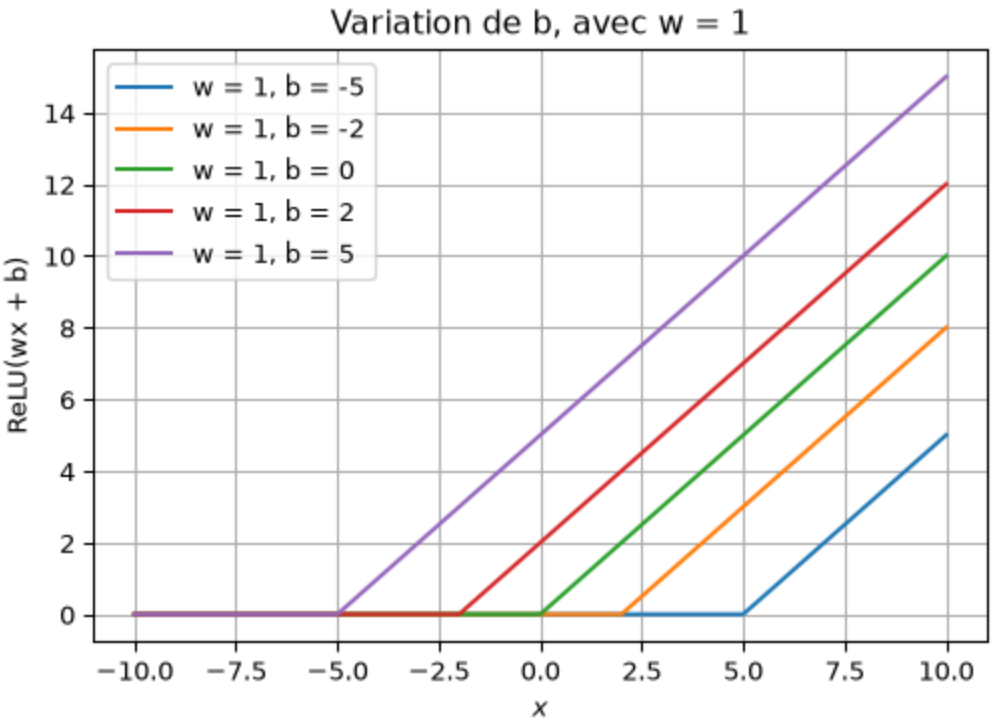
In [10]:
```
plot_parameter_effect(relu, "ReLU", weights=[0.5, 1, 2, 5], biases=[0],
                      title="Variation de w, avec b = 0")
```



Le poids modifie la **pente** de la partie active de $\mathrm{ReLU}(wx + b)$.

Effet de b : ReLU

In [11]:
```
plot_parameter_effect(relu, "ReLU", weights=[1], biases=[-5, -2, 0, 2, 5],
                      title="Variation de b, avec w = 1")
```



Pour $w \neq 0$ fixé, le biais déplace le **changement de pente** en $x = -b/w$.

Complément interactif : Construire une fonction avec deux unités ReLU.

Démonstration par le code

Approximer la même courbe cible avec des réseaux ayant des **nombres différents d'unités cachées**.

In [12]:
```
# Définition de la fonction à approximer

def f(x):
    return 2 * x**3 + 4 * x**2 - 5 * x + 1

# Génération des données : x dans [-4,2), avec f(x) définie ci-dessus

approximation_rng = np.random.default_rng(42)
X = approximation_rng.uniform(-4, 2, size=(1000, 1))

y = f(X.flatten())
```

$y = f(x) = 2x^3 + 4x^2 - 5x + 1$

Augmenter le nombre de neurones

In [13]:
```
from sklearn.neural_network import MLPRegressor
from sklearn.model_selection import train_test_split
```

```
X_train, X_valid, y_train, y_valid = train_test_split(
    X, y, test_size=0.1, random_state=42
)

models = []

sizes = [1, 2, 5, 10, 100]

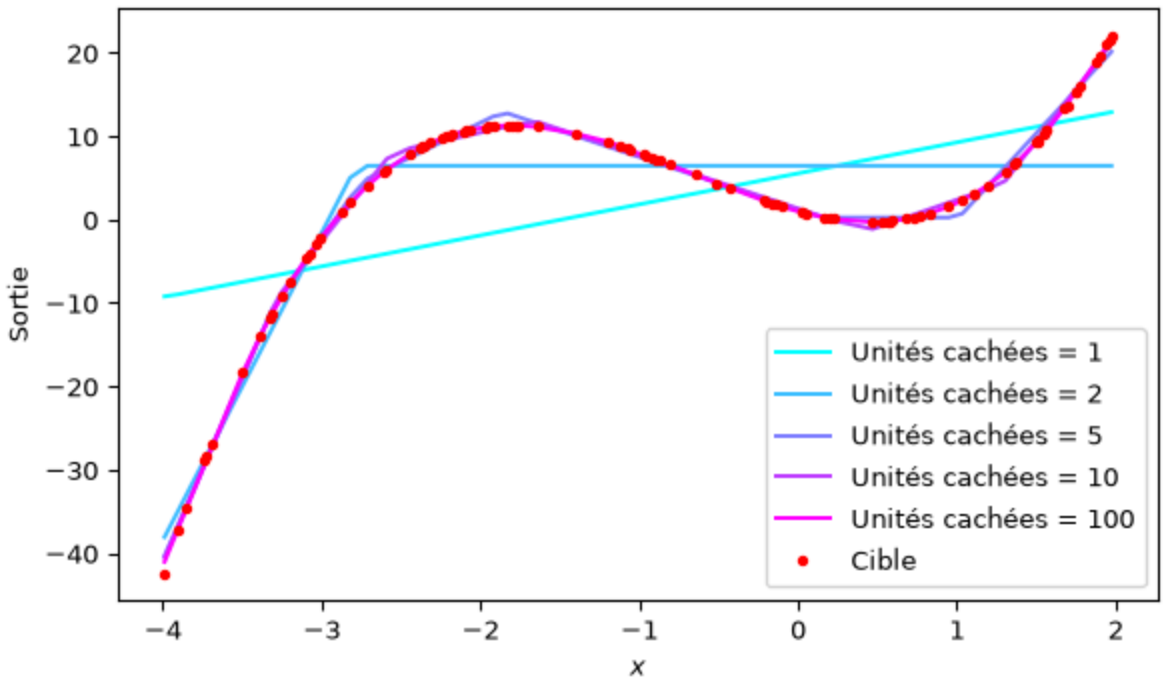
for n in sizes:
    regressor = MLPRegressor(
        hidden_layer_sizes=(n,), max_iter=5000, random_state=42
    )
    regressor.fit(X_train, y_train)
    models.append(regressor)
```

MLPRegressor est un régresseur à perceptron multicouche de sklearn. Sa fonction d'activation par défaut est `relu`.

Comparaison des courbes ajustées

In [14]:
```
colors = plt.colormaps["cool"].resampled(len(sizes))
plot_inputs = np.sort(X_valid, axis=0)

fig, ax = plt.subplots(figsize=(7, 4))
for i, n in enumerate(sizes):
    predictions = models[i].predict(plot_inputs)
    ax.plot(plot_inputs, predictions, color=colors(i),
            label=f"Unités cachées = {n}")
ax.plot(plot_inputs, f(plot_inputs), "r.", label="Cible")
ax.set(xlabel="$x$", ylabel="Sortie")
ax.legend()
plt.show()
```



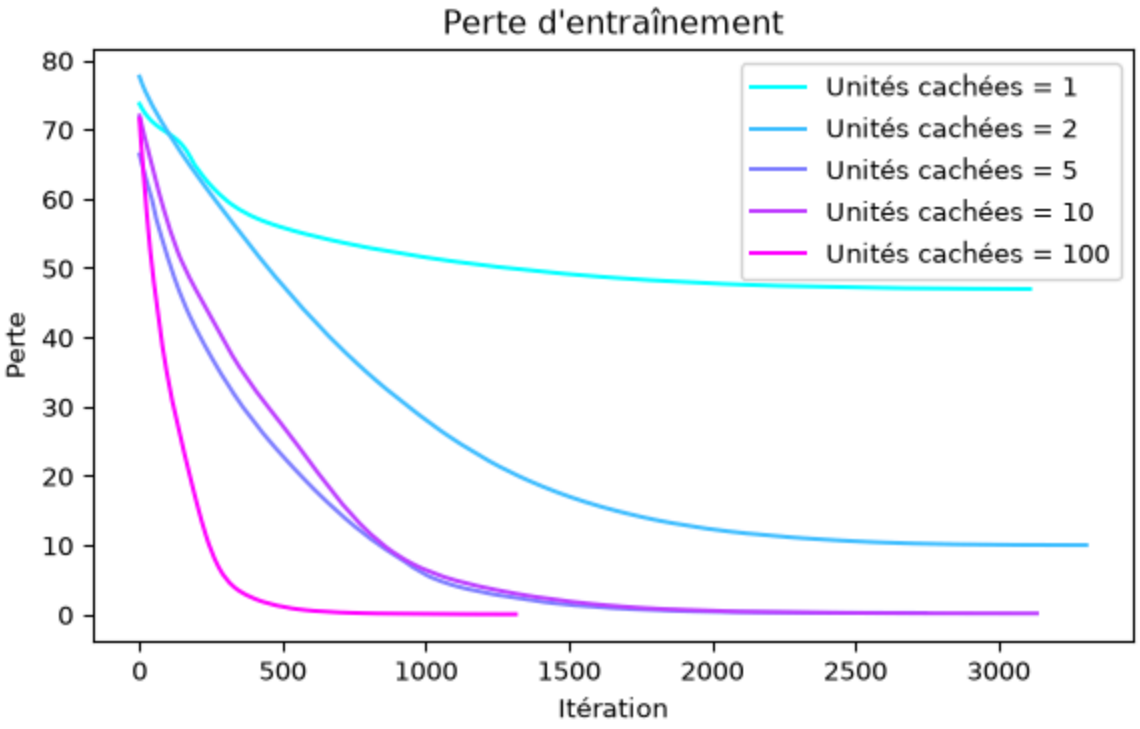
La cible est une fonction cubique connue, sans bruit. Chaque modèle possède une couche cachée; modifier son nombre d'unités change les fonctions que le modèle peut représenter. Le sous-ensemble mis de côté est un **ensemble de validation**, utilisé ici pour visualiser les prédictions en dehors des exemples d'entraînement.

Les courbes montrent les réseaux obtenus par une procédure d'entraînement particulière. Elles illustrent l'approximation en pratique, mais ne prouvent pas le théorème : un réseau entraîné peut ne pas trouver une bonne approximation, même lorsque des paramètres appropriés existent. Un plus grand nombre d'unités cachées ne produit pas nécessairement une meilleure courbe ajustée à chaque exécution.

Pour une explication visuelle complémentaire, voir le chapitre de Michael Nielsen sur l'approximation universelle.

Comparaison des pertes d'entraînement

In [15]:
```
fig, ax = plt.subplots(figsize=(7, 4))
for i, n in enumerate(sizes):
    ax.plot(models[i].loss_curve_, color=colors(i),
            label=f"Unités cachées = {n}")
ax.set(title="Perte d'entraînement", xlabel="Itération", ylabel="Perte")
ax.legend()
plt.show()
```



Davantage d'unités cachées augmentent la **capacité de représentation**. La perte atteinte par l'entraînement dépend aussi de l'**optimisation**.

Ces courbes représentent les pertes d'entraînement. Des réseaux initialisés et entraînés indépendamment n'atteignent pas nécessairement des pertes ordonnées selon leur nombre d'unités cachées. Leurs pertes finales reflètent aussi la procédure d'optimisation et la régularisation utilisée par l'estimateur. Le théorème garantit l'existence d'une approximation sous ses hypothèses, et non la réussite de ces entraînements particuliers.

Approximation universelle

https://youtu.be/CqOfi41LfDw

Cette vidéo présente visuellement l'intuition du théorème d'approximation universelle. (18 min 53 s)

La vidéo fournit une explication visuelle complémentaire de la manière dont les sorties pondérées des activations peuvent être combinées pour approximer une courbe cible. Un poids de sortie multiplie la contribution d'une unité cachée et peut en inverser le signe. Un biais d'entrée à l'intérieur d'une activation, comme $\sigma(wx + b)$, déplace sa transition horizontalement lorsque $w \neq 0$; un biais à une sortie linéaire déplace la courbe combinée verticalement.

Codons

Bibliothèques et environnements

PyTorch, TensorFlow et JAX fournissent des outils de calcul numérique et de différentiation automatique.

Keras fournit une API de haut niveau pour construire et entraîner des modèles avec ces moteurs de calcul (*backends*).

Un environnement de calcul fournit des opérations permettant de calculer efficacement les sorties des modèles et leurs gradients. Une API de haut niveau propose des interfaces familières pour définir un modèle, l'ajuster à des données, l'évaluer et produire des prédictions.

L'exemple utilise Keras par l'intermédiaire de `tf.keras`, avec TensorFlow comme moteur de calcul. La distinction entre propagation avant, calcul des gradients et mises à jour par l'optimiseur reste la même d'un environnement à l'autre. Premiers pas avec Keras

Keras

Les principales étapes sont :

1. **Définir** les couches du réseau.
2. **Compiler** avec une perte, un optimiseur et des métriques d'évaluation.
3. **Ajuster** le modèle aux données d'entraînement et suivre ses performances sur l'ensemble de validation.
4. **Évaluer** le modèle sur l'ensemble de test et produire des prédictions.

Keras a été créé par François Chollet. Son API Sequential décrit une succession de couches, chacune recevant la sortie de la couche précédente. Elle convient à ce classificateur à propagation avant.

Le code qui suit forme une procédure complète lorsqu'il est exécuté dans l'ordre. Le softmax et l'entropie croisée catégorielle sont introduits ici par leur rôle dans l'API; leurs détails mathématiques sont développés dans la présentation 14.

Jeu de données Fashion-MNIST

Fashion-MNIST contient des images de vêtements et d'accessoires :

- **60 000 images d'entraînement** et **10 000 images de test**.
- Chaque image comporte **28 × 28 pixels en niveaux de gris**.
- Chaque image possède **une étiquette entière**, parmi **10 classes**.

Exemple adapté de Géron (2022), cahier du chapitre 10 de *Hands-On Machine Learning*.

Chargement et partitionnement

In [16]:
```
import tensorflow as tf

(X_train_full, y_train_full), (X_test, y_test) = (
    tf.keras.datasets.fashion_mnist.load_data()
)

X_train, y_train = X_train_full[:-5000], y_train_full[:-5000]
X_valid, y_valid = X_train_full[-5000:], y_train_full[-5000:]
```

Utilisez **55 000** exemples pour l'entraînement et **5 000** pour la validation. Réservez l'**ensemble de test** fourni à l'évaluation finale.

Les exemples de validation proviennent de la partition d'entraînement fournie. Ils servent à suivre les performances pendant le développement, mais pas à mettre à jour les paramètres du modèle. La partition de test reste distincte de l'entraînement et de la sélection du modèle, comme discuté dans la présentation 8.

Inspection et mise à l'échelle des entrées

In [17]:
```
X_train.shape, X_train.dtype
```

```
((55000, 28, 28), dtype('uint8'))
```

...

Convertissez les intensités des pixels, des **entiers de 0 à 255**, en **nombres flottants de 0 à 1**, dans chaque partition.

In [18]:
```
X_train = X_train.astype("float32") / 255.0
X_valid = X_valid.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0
```

Le modèle doit recevoir des entrées à la même échelle pendant l'entraînement, la validation, le test et les prédictions ultérieures. Utiliser des pixels de test bruts entre 0 et 255 après un entraînement avec des pixels entre 0 et 1 modifie les entrées de chaque somme pondérée et invalide la comparaison.

La division par la borne connue de 255 n'estime aucune statistique à partir des données. En revanche, un prétraitement qui apprend une moyenne, une variance ou une autre statistique doit être ajusté sur les données d'entraînement, puis appliqué aux autres partitions à l'aide de la transformation ajustée.

Images et étiquettes

In [19]:
```
class_names = ["T-shirt/haut", "Pantalon", "Pull", "Robe", "Manteau",
               "Sandale", "Chemise", "Espadrille", "Sac", "Bottine"]

plt.figure(figsize=(2, 2))
plt.imshow(X_train[0], cmap="binary")
plt.title(class_names[y_train[0]])
plt.axis("off")
plt.show()
```



La cible est un indice de classe entier; `class_names` fournit une étiquette lisible.

Les 40 premières images

In [20]:
```
fig, axes = plt.subplots(4, 10, figsize=(12, 4.8))
for index, ax in enumerate(axes.flat):
    ax.imshow(X_train[index], cmap="binary", interpolation="nearest")
    ax.set_title(class_names[y_train[index]], fontsize=10)
    ax.axis("off")
plt.tight_layout()
plt.show()
```



Création d'un modèle

In [21]:
```
tf.keras.utils.set_random_seed(42)

model = tf.keras.Sequential([
    tf.keras.Input(shape=(28, 28)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(300, activation="relu"),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])
```

Input indique la forme d'un exemple. Flatten réorganise les 28 × 28 pixels en 784 attributs d'entrée sans apprendre de paramètres. Les deux couches Dense cachées calculent des sommes pondérées plus des biais, suivies d'activations ReLU.

La dernière couche Dense produit dix scores et applique softmax pour former une distribution de probabilités sur les dix classes mutuellement exclusives. La somme de ces probabilités vaut 1. Les activations cachées et les activations de sortie jouent des rôles différents.

L'utilitaire de graine aléatoire fixe les graines de Python, NumPy et TensorFlow pour favoriser la reproductibilité; les résultats numériques exacts peuvent encore dépendre de l'environnement d'exécution. Utilitaire de graine aléatoire de Keras

Structure du modèle et paramètres

In [22]:
```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Par
flatten (Flatten)	(None, 784)	
dense (Dense)	(None, 300)	235
dense_1 (Dense)	(None, 100)	30
dense_2 (Dense)	(None, 10)	1

Total params: 266,610 (1.02 MB)
Trainable params: 266,610 (1.02 MB)
Non-trainable params: 0 (0.00 B)

Une couche Dense avec n_{in} entrées et n_{out} unités possède $(n_{\text{in}} + 1)n_{\text{out}}$ paramètres entraînables, biais compris.

Il y a un poids par connexion entre une entrée et une unité, et un biais par unité. La première couche cachée possède donc $784 \times 300 + 300 = 235{,}500$ paramètres. La deuxième en possède $300 \times 100 + 100 = 30{,}100$, et la couche de sortie, $100 \times 10 + 10 = 1{,}010$. Le total est de $266{,}610$ paramètres entraînables. Flatten n'en possède aucun.

Pour un lot de m exemples, la première couche cachée reçoit un tableau de taille $m \times 784$, utilise une matrice de poids de taille 784×300 et un vecteur de 300 biais, puis produit une sortie de taille $m \times 300$. Cela suit la convention précédente : une ligne par exemple et une colonne par unité dans la matrice des poids.

Compilation du modèle

In [23]:
```
model.compile(
    loss="sparse_categorical_crossentropy",
    optimizer=tf.keras.optimizers.SGD(learning_rate=0.01),
    metrics=["accuracy"]
)
```

Perte : comparer la distribution prédite à l'étiquette cible entière.

Optimiseur : mettre à jour les paramètres à l'aide des gradients. **Exactitude :** rapporter la proportion de prédictions correctes.

L'entropie croisée catégorielle avec étiquettes entières (`sparse_categorical_crossentropy`) convient parce que chaque exemple possède une seule étiquette de classe entière et que le modèle produit une distribution

de probabilités catégorielle. Avec des cibles encodées en vecteurs indicateurs (*one-hot*), la perte correspondante est l'entropie croisée catégorielle. Ici, la perte reçoit les probabilités issues de softmax, plutôt que les scores bruts (*logits*). Pertes probabilistes de Keras

La méthode `compile` configure l'entraînement; elle n'ajuste pas les paramètres. La descente de gradient stochastique utilise les gradients calculés par rétropropagation pour mettre à jour les poids et les biais. Le taux d'apprentissage règle l'amplitude de ces mises à jour. L'exactitude (*accuracy*) est rapportée comme métrique, tandis que la perte différentiable fournit l'objectif d'entraînement.

Entraînement du modèle

In [24]:
```
history = model.fit(
    X_train, y_train, epochs=30, batch_size=32,
    validation_data=(X_valid, y_valid), verbose=2
)
```

```
Epoch 1/30
1719/1719 - 2s - 910us/step - accuracy: 0.7616 - loss: 0.7239 - val_accurac
y: 0.8298 - val_loss: 0.5026
Epoch 2/30
1719/1719 - 1s - 725us/step - accuracy: 0.8299 - loss: 0.4894 - val_accurac
y: 0.8418 - val_loss: 0.4472
Epoch 3/30
1719/1719 - 1s - 736us/step - accuracy: 0.8448 - loss: 0.4434 - val_accurac
y: 0.8480 - val_loss: 0.4233
Epoch 4/30
1719/1719 - 1s - 750us/step - accuracy: 0.8548 - loss: 0.4154 - val_accurac
y: 0.8550 - val_loss: 0.4088
Epoch 5/30
1719/1719 - 1s - 744us/step - accuracy: 0.8607 - loss: 0.3951 - val_accurac
y: 0.8584 - val_loss: 0.3970
Epoch 6/30
1719/1719 - 1s - 750us/step - accuracy: 0.8667 - loss: 0.3790 - val_accurac
y: 0.8594 - val_loss: 0.3889
Epoch 7/30
1719/1719 - 1s - 836us/step - accuracy: 0.8713 - loss: 0.3656 - val_accurac
y: 0.8616 - val_loss: 0.3824
Epoch 8/30
1719/1719 - 2s - 879us/step - accuracy: 0.8748 - loss: 0.3539 - val_accurac
y: 0.8642 - val_loss: 0.3780
Epoch 9/30
1719/1719 - 1s - 847us/step - accuracy: 0.8783 - loss: 0.3432 - val_accurac
y: 0.8666 - val_loss: 0.3740
Epoch 10/30
1719/1719 - 1s - 736us/step - accuracy: 0.8817 - loss: 0.3336 - val_accurac
y: 0.8678 - val_loss: 0.3684
Epoch 11/30
1719/1719 - 1s - 737us/step - accuracy: 0.8851 - loss: 0.3247 - val_accurac
y: 0.8702 - val_loss: 0.3641
Epoch 12/30
1719/1719 - 1s - 749us/step - accuracy: 0.8875 - loss: 0.3165 - val_accurac
y: 0.8726 - val_loss: 0.3601
Epoch 13/30
1719/1719 - 1s - 729us/step - accuracy: 0.8898 - loss: 0.3089 - val_accurac
y: 0.8736 - val_loss: 0.3578
Epoch 14/30
1719/1719 - 1s - 730us/step - accuracy: 0.8922 - loss: 0.3017 - val_accurac
y: 0.8724 - val_loss: 0.3554
Epoch 15/30
1719/1719 - 1s - 735us/step - accuracy: 0.8942 - loss: 0.2951 - val_accurac
y: 0.8736 - val_loss: 0.3543
Epoch 16/30
1719/1719 - 1s - 743us/step - accuracy: 0.8962 - loss: 0.2887 - val_accurac
y: 0.8734 - val_loss: 0.3520
Epoch 17/30
1719/1719 - 1s - 738us/step - accuracy: 0.8983 - loss: 0.2829 - val_accurac
y: 0.8738 - val_loss: 0.3522
Epoch 18/30
1719/1719 - 1s - 732us/step - accuracy: 0.9002 - loss: 0.2771 - val_accurac
y: 0.8746 - val_loss: 0.3495
Epoch 19/30
1719/1719 - 1s - 749us/step - accuracy: 0.9021 - loss: 0.2717 - val_accurac
```

```
y: 0.8758 - val_loss: 0.3478
Epoch 20/30
1719/1719 - 1s - 755us/step - accuracy: 0.9043 - loss: 0.2664 - val_accurac
y: 0.8760 - val_loss: 0.3465
Epoch 21/30
1719/1719 - 1s - 729us/step - accuracy: 0.9060 - loss: 0.2615 - val_accurac
y: 0.8754 - val_loss: 0.3465
Epoch 22/30
1719/1719 - 1s - 743us/step - accuracy: 0.9081 - loss: 0.2565 - val_accurac
y: 0.8772 - val_loss: 0.3449
Epoch 23/30
1719/1719 - 1s - 724us/step - accuracy: 0.9102 - loss: 0.2519 - val_accurac
y: 0.8782 - val_loss: 0.3454
Epoch 24/30
1719/1719 - 1s - 719us/step - accuracy: 0.9121 - loss: 0.2471 - val_accurac
y: 0.8780 - val_loss: 0.3445
Epoch 25/30
1719/1719 - 1s - 744us/step - accuracy: 0.9145 - loss: 0.2427 - val_accurac
y: 0.8790 - val_loss: 0.3437
Epoch 26/30
1719/1719 - 1s - 727us/step - accuracy: 0.9158 - loss: 0.2384 - val_accurac
y: 0.8792 - val_loss: 0.3446
Epoch 27/30
1719/1719 - 1s - 735us/step - accuracy: 0.9172 - loss: 0.2341 - val_accurac
y: 0.8810 - val_loss: 0.3445
Epoch 28/30
1719/1719 - 1s - 764us/step - accuracy: 0.9193 - loss: 0.2299 - val_accurac
y: 0.8804 - val_loss: 0.3441
Epoch 29/30
1719/1719 - 1s - 737us/step - accuracy: 0.9210 - loss: 0.2260 - val_accurac
y: 0.8808 - val_loss: 0.3445
Epoch 30/30
1719/1719 - 1s - 745us/step - accuracy: 0.9223 - loss: 0.2220 - val_accurac
y: 0.8800 - val_loss: 0.3440
```

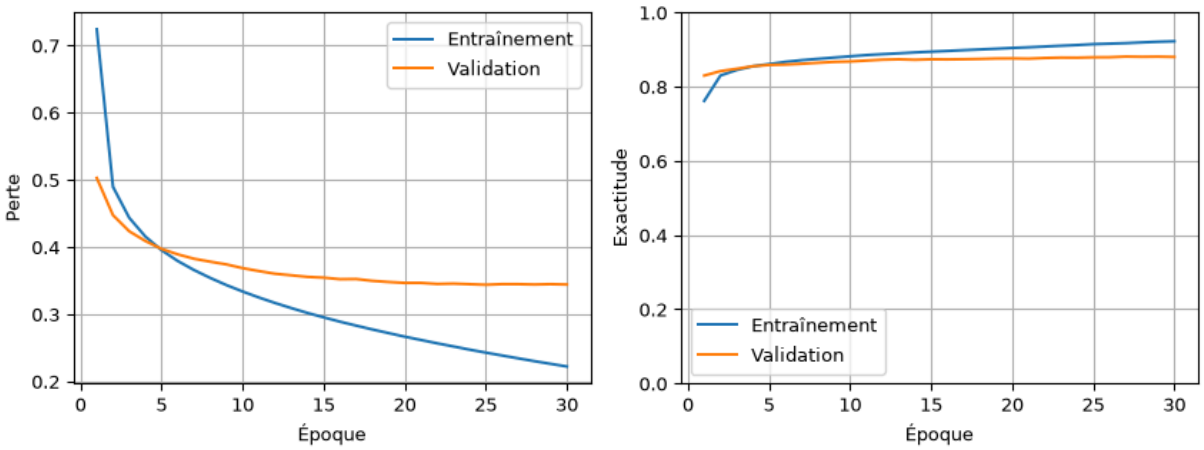
Une étape d'entraînement traite un mini-lot et met les paramètres à jour une fois. Une époque correspond à un passage complet sur les exemples d'entraînement. La taille du lot et le taux d'apprentissage relient cet exemple aux méthodes de descente de gradient stochastique et par mini-lots introduites dans la présentation 4.

Avec cet appel, Keras évalue l'ensemble de validation à la fin de chaque époque, sans y mettre à jour les paramètres. L'objet History retourné conserve la perte et l'exactitude d'entraînement et de validation pour chaque époque. API d'entraînement de Keras

Les métriques d'entraînement agrègent les résultats des mini-lots alors que les paramètres changent au cours de l'époque; les métriques de validation utilisent les paramètres de fin d'époque. Leurs courbes ne sont donc pas évaluées exactement dans les mêmes conditions. Les résultats de validation peuvent guider le choix du modèle et des hyperparamètres; l'ensemble de test est réservé à l'évaluation finale.

Courbes d'entraînement et de validation

In [25]:
```
epochs = np.arange(1, len(history.history["loss"]) + 1)
fig, axes = plt.subplots(1, 2, figsize=(9, 3.5))
for ax, metric, title in zip(
    axes, ["loss", "accuracy"], ["Perte", "Exactitude"]
):
    ax.plot(epochs, history.history[metric], label="Entraînement")
    ax.plot(epochs, history.history[f"val_{metric}"], label="Validation")
    ax.set(xlabel="Époque", ylabel=title)
    ax.grid(True)
    ax.legend()
axes[1].set_ylim(0, 1)
plt.tight_layout()
plt.show()
```



La perte et l'exactitude décrivent des aspects différents des performances et ont des échelles différentes. L'exactitude est comprise entre 0 et 1, mais la perte d'entropie croisée peut dépasser 1 : l'axe de la perte n'est donc pas limité à cet intervalle.

Une diminution continue de la perte d'entraînement accompagnée d'une dégradation de la perte de validation peut signaler du surapprentissage (*overfitting*). Des fluctuations isolées ne suffisent pas à établir une tendance, et l'entraînement ne garantit pas une amélioration de la validation à chaque époque. La régularisation et l'arrêt précoce (*early stopping*) sont examinés dans la présentation 14.

Évaluation finale sur l'ensemble de test

In [26]:
```
test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f"Perte de test : {test_loss:.3f}")
print(f"Exactitude de test : {test_accuracy:.3f}")
```

```
Perte de test : 0.362
Exactitude de test : 0.874
```

L'ensemble de test estime les performances sur des exemples mis de côté, une fois le modèle et sa configuration d'entraînement choisis. Ses entrées reçoivent la même mise à l'échelle que les entrées d'entraînement et de validation. Les résultats de test ne doivent pas servir à choisir l'architecture ou le nombre d'époques.

Faire des prédictions

In [27]:
```
X_new = X_test[:3]
y_proba = model.predict(X_new, verbose=0)
y_proba.round(2)
```

```
array([[0.  , 0.  , 0.  , 0.  , 0.  , 0.33, 0.  , 0.01, 0.  , 0.65],
       [0.  , 0.  , 1.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  ],
       [0.  , 1.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  , 0.  ]],
      dtype=float32)
```

...

In [28]:
```
y_pred = y_proba.argmax(axis=1)
y_pred
```

```
array([9, 2, 1])
```

...

In [29]:
```
y_observed = y_test[:3]
y_observed
```

```
array([9, 2, 1], dtype=uint8)
```

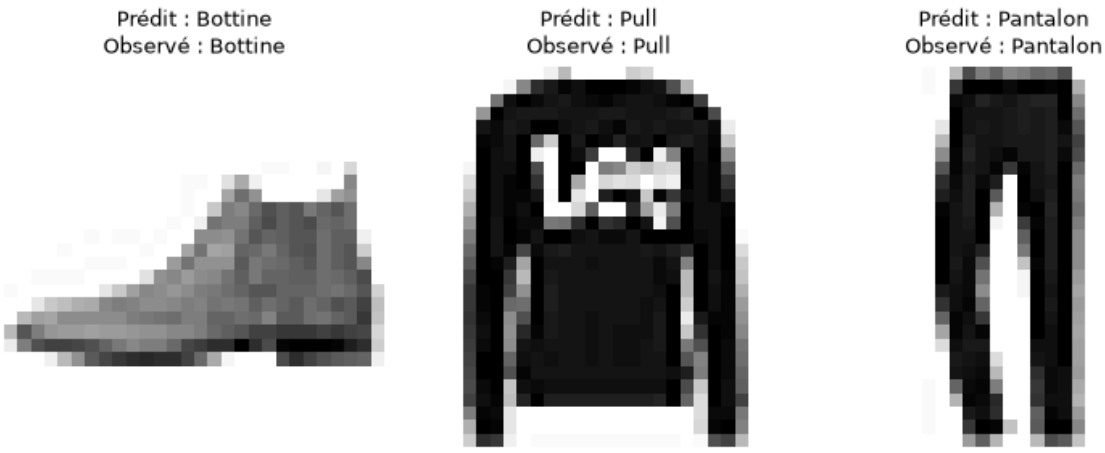
La classe prédite possède la **probabilité de sortie la plus élevée**. Comparez-la à la **classe observée**.

Chaque ligne de `y_proba` contient dix probabilités de classe pour une image. `argmax(axis=1)` choisit l'indice de la plus grande probabilité dans chaque ligne. Ces indices entiers correspondent aux entrées de `class_names`.

Une probabilité de sortie élevée exprime la confiance du modèle dans sa prédiction; elle n'établit pas que la prédiction est correcte. Les étiquettes observées sont nécessaires pour évaluer ces exemples.

Étiquettes prédites et observées

In [30]:
```
fig, axes = plt.subplots(1, 3, figsize=(8, 3))
for ax, image, prediction, observed in zip(axes, X_new, y_pred, y_observed):
    ax.imshow(image, cmap="binary", interpolation="nearest")
    ax.set_title(f"Prédit : {class_names[prediction]}\n"
                 f"Observé : {class_names[observed]}", fontsize=10)
    ax.axis("off")
plt.tight_layout()
plt.show()
```



Performances par classe

In [31]:
```
from sklearn.metrics import classification_report

y_test_proba = model.predict(X_test, verbose=0)
y_test_pred = y_test_proba.argmax(axis=1)
print(classification_report(y_test, y_test_pred,
                            target_names=class_names, digits=3))
```

```
              precision    recall  f1-score   support

T-shirt/haut      0.842     0.810     0.826      1000
    Pantalon      0.992     0.964     0.978      1000
        Pull      0.744     0.850     0.793      1000
        Robe      0.824     0.929     0.873      1000
     Manteau      0.806     0.794     0.800      1000
     Sandale      0.885     0.986     0.933      1000
     Chemise      0.777     0.618     0.689      1000
  Espadrille      0.944     0.911     0.927      1000
         Sac      0.961     0.963     0.962      1000
     Bottine      0.983     0.918     0.949      1000

    accuracy                          0.874     10000
   macro avg      0.876     0.874     0.873     10000
weighted avg      0.876     0.874     0.873     10000
```

L'exactitude globale peut masquer des différences entre les classes. Le rapport de classification donne la précision, le rappel, le score F1 et l'effectif de chaque catégorie de vêtements, selon les définitions introduites dans la présentation 7. Les noms de classes lisibles aident à repérer les catégories les plus difficiles pour ce modèle.

Conclusion

Résumé

- **Calcul :** la propagation avant associe des sorties aux entrées à l'aide des poids, des biais et des activations.
- **Représentation :** des couches cachées non linéaires calculent des attributs intermédiaires qui peuvent rendre XOR séparable.
- **Apprentissage :** l'entraînement ajuste les paramètres; la rétropropagation calcule les gradients utilisés par un optimiseur.
- **Approximation universelle :** des paramètres appropriés peuvent exister sans garantie que l'entraînement les trouvera.
- **Implémentation :** Keras relie ces idées dans une procédure complète de classification sur Fashion-MNIST.
- **Évaluation :** appliquer un prétraitement cohérent et distinguer les performances d'entraînement, de validation et de test.

3Blue1Brown sur l'apprentissage profond

- But what is a Neural Network?
  - youtu.be/aircAruvnKk (19 minutes)
- Gradient descent, how neural networks learn?
  - youtu.be/IHZwWFHWa-w (21 minutes)
- What is backpropagation really doing?
  - youtu.be/Ilg3gGewQ5U (14 minutes)
- Backpropagation calculus
  - youtu.be/tIeHLnjs5U8 (10 minutes)

Prochaine présentation

- Nous examinerons l'implémentation d'un réseaux de neurones artificiels.

Références

Cybenko, George V. 1989. « Approximation by superpositions of a sigmoidal function ». *Mathematics of Control, Signals and Systems* 2: 303-14. https://api.semanticscholar.org/CorpusID:3958369.

Géron, Aurélien. 2022. *Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3e éd. O'Reilly Media, Inc.

Goodfellow, Ian, Yoshua Bengio, et Aaron Courville. 2016. *Deep Learning*. Adaptive computation et machine learning. MIT Press. https://dblp.org/rec/books/daglib/0040158.

Hornik, Kurt, Maxwell Stinchcombe, et Halbert White. 1989. « Multilayer feedforward networks are universal approximators ». *Neural Networks* 2 (5): 359-66. https://doi.org/https://doi.org/10.1016/0893-6080(89)90020-8.

Lakoff, George, et Srini Narayanan. 2025. *The Neural Mind: How Brains Think*. 1re éd. University of Chicago Press. https://press.uchicago.edu/ucp/books/book/chicago/N/bo243406239.html.

LeCun, Yann, Yoshua Bengio, et Geoffrey Hinton. 2015. « Deep learning ». *Nature* 521 (7553): 436-44. https://doi.org/10.1038/nature14539.

LeNail, Alexander. 2019. « NN-SVG: Publication-Ready Neural Network Architecture Schematics ». *Journal of Open Source Software* 4 (33): 747. https://doi.org/10.21105/joss.00747.

McCulloch, Warren S, et Walter Pitts. 1943. « A logical calculus of the ideas immanent in nervous activity ». *The Bulletin of Mathematical Biophysics* 5 (4): 115-33. https://doi.org/10.1007/bf02478259.

Minsky, Marvin, et Seymour Papert. 1969. *Perceptrons: An Introduction to Computational Geometry*. MIT Press.

Rosenblatt, F. 1958. « The perceptron: A probabilistic model for information storage and organization in the brain. » *Psychological Review* 65 (6): 386-408. https://doi.org/10.1037/h0042519.

Rumelhart, David E., Geoffrey E. Hinton, et Ronald J. Williams. 1986. « Learning representations by back-propagating errors ». *Nature* 323 (6088): 533-36. https://doi.org/10.1038/323533a0.

Russell, Stuart, et Peter Norvig. 2020. *Artificial Intelligence: A Modern Approach*. 4e éd. Pearson. http://aima.cs.berkeley.edu/.

Marcel **Turcotte**

Marcel.Turcotte@uOttawa.ca

École de **science informatique** et de génie électrique (**SI**GE)

Université d'Ottawa